CURSO DE FORMACIÓN · 20 HORAS

Big Data e IA

Bloques 1 y 2: fundamentos y tratamiento de datos



5 horas · Python · pandas · Google Colab

PRESENTACIÓN

Quién os va a acompañar

Pablo Mesa Canal

Senior Platform Architect

Zscaler

EXPERIENCIA

- [Puesto · empresa · años]
- [Puesto · empresa · años]

FORMACIÓN

Ingeniero en Informática

ESPECIALIDADES

BigData · IA · Ciberseguridad

PLAN DE LA SESIÓN

Cinco horas, dos bloques

BLOQUE 1 · 2 H

Fundamentos de Big Data

- Las V del Big Data y computación distribuida
- Tipos de datos y modelos de datos
- Consistencia, almacenamiento y arquitecturas (streaming, kappa, lambda, medallion)
- Estrategia y seguridad del dato
- Ética, normativa y calidad del dato

BLOQUE 2 · 3 H

Programación para datos

- Python con pandas y NumPy
- Carga, limpieza y transformación
- Caso práctico en Google Colab

01

Fundamentos de Big Data

Bloque 1 · 2 horas

> «Es un error capital teorizar antes de tener datos.»
>
> Sherlock Holmes, en *Escándalo en Bohemia*, de Arthur Conan Doyle (1891)

> «Los datos son el nuevo petróleo.»
>
> Clive Humby, matemático y pionero de la analítica de clientes (2006)

PLAN DEL BLOQUE

Hoja de ruta del Bloque 1

1 — 1.1–1.2 — **Qué es Big Data y las V**

2 — 1.3 — **Retos y cómputo distribuido**

3 — 1.4 — **Tipos de datos y bases de datos**

4 — 1.5 — **Modelos de datos**

5 — 1.6 — **Consistencia y almacenamiento**

6 — 1.7 — **Arquitectura y streaming**

7 — 1.8 — **Estrategia de datos**

8 — 1.9 — **Seguridad del dato**

9 — 1.10–1.11 — **Ética, normativa y calidad**

Del dato en bruto a **decidir con confianza**: primero qué es y cuánto pesa, luego dónde y cómo se guarda, y al final cómo se protege, se usa con ética y se comprueba su calidad.

1.1 · CONCEPTO

¿Qué es Big Data?

Datos tan grandes, rápidos o variados que las herramientas tradicionales no pueden almacenarlos ni procesarlos con eficiencia.

- No caben en una sola máquina
- Llegan más rápido de lo que se procesan
- Vienen de fuentes y formatos muy distintos

1.2 · CARACTERÍSTICAS

Las V del Big Data

V

Volumen

Cantidad masiva de datos, de gigabytes a petabytes.

Millones de compras al día

V

Velocidad

Ritmo al que se generan y deben procesarse.

Sensores y streaming en tiempo real

V

Variedad

Formatos y fuentes muy distintos entre sí.

Tablas, JSON, texto, imágenes

V

Veracidad

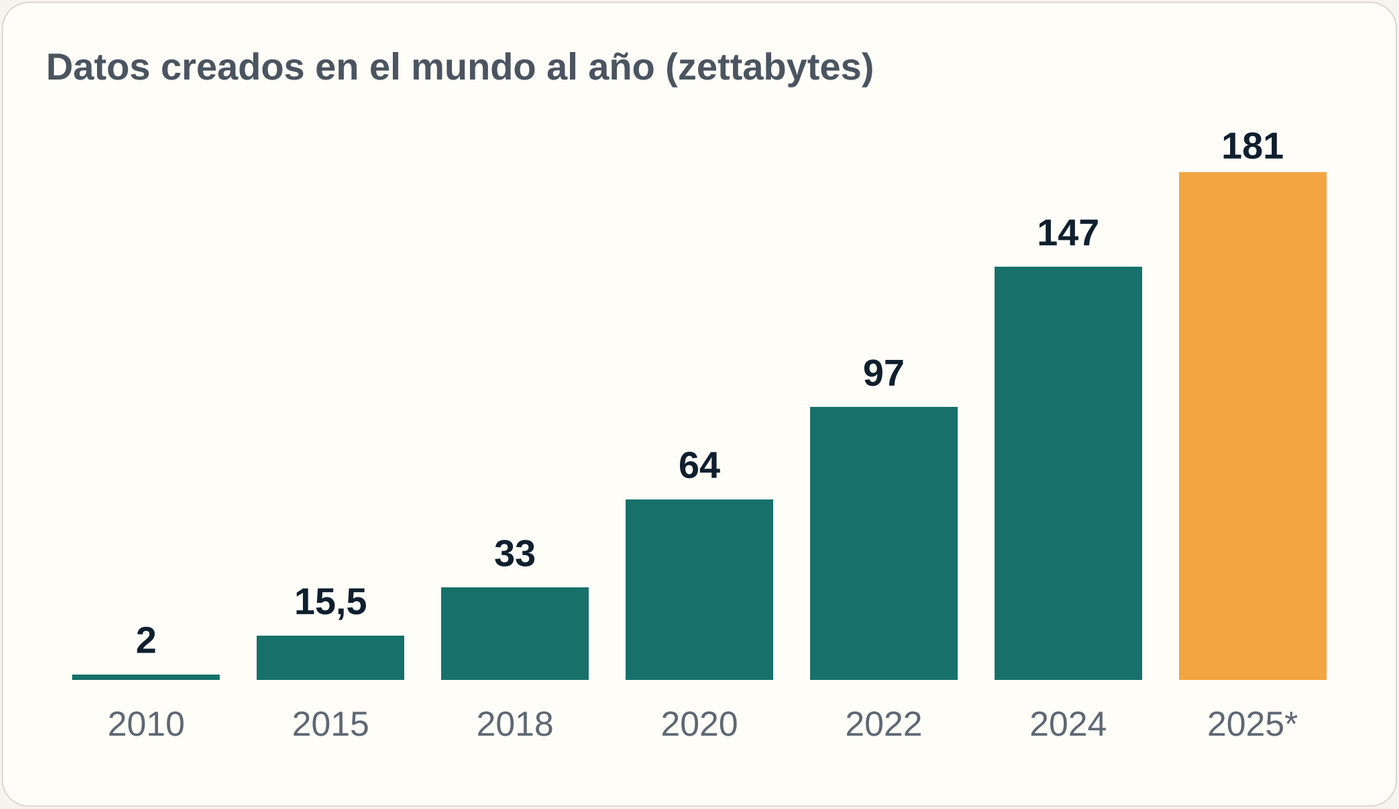
Calidad y fiabilidad del dato.

Nulos, duplicados, errores

A veces se añade una quinta V: **Valor**, convertir los datos en decisiones útiles.

1.2 · VOLUMEN

Cada año se crean más datos que el anterior



DE 2010 A 2025*

×90

de 2 a unos 181 zettabytes al año.

EN NUESTRO CASO

38 partidos del Sporting son 128.813 eventos. Una temporada completa de 380 partidos serían más de 1,2 millones.

Cuando el volumen se convierte en valor

CIENCIA

CERN: guardar solo lo que importa

Todas las colisiones del LHC

Filtro rápido en hardware

Filtro en software

≈ 1 de cada 3 millones se conserva

≈ 20 PB al año. El análisis del bosón de Higgs manejó varios cientos de PB.

STREAMING

Netflix: recomendar a millones

DATO

El comportamiento de millones de suscriptores.

DECISIÓN

Qué mostrar a cada persona.

RESULTADO

Las recomendaciones personalizadas se elegían 3–4 veces más que una lista de populares. Sus directivos estimaron un ahorro de ≈ 1.000 M$ al año en bajas.

EN NUESTRO CASO 128.813 eventos caben en un portátil. La lógica de filtrar y agregar es la misma que en el CERN.

1.2 · VELOCIDAD

Un coche de Fórmula 1 en tiempo real

300

sensores en cada coche

1,1 M

datos por segundo y coche

66 millones

de datos por minuto, por cada coche

≈ 5.900 millones

en una carrera de 90 minutos, por coche

≈ 0,6 por segundo

eventos que registra un partido de fútbol

Los datos llegan al muro de boxes mientras el coche corre: **la estrategia se decide en directo**.

De los sensores a la decisión

Coche
300 sensores

→

Boxes
1,1 M datos por segundo

→

Streaming
los datos fluyen según se generan

→

Modelos
predicen en tiempo real

→

Decisión
¿paramos ahora?

UNDERCUT THREAT

Avisa de qué coche corre riesgo de ser rebasado por una parada, antes de que ninguno pare.

PITLANE PERFORMANCE

Analiza cada paso de la parada y el tiempo total perdido en el pit lane.

CAR EXPLOITATION

Compara el rendimiento del coche con su límite teórico, vuelta a vuelta.

Los análisis de **F1 Insights** son los que ve la afición durante la carrera; los equipos tienen los suyos.

Un dato tarde es un dato inútil

Por lotes
(batch)

El dato llega → Se guarda → Se procesa más tarde → Informe al día siguiente

En continuo
(streaming)

El dato llega → Se procesa al llegar → Alerta o decisión al momento

Mismo dato, otro momento: el desgaste de un neumático a mitad de carrera solo sirve si llega a tiempo.

Cuando la rapidez cambia la decisión

PAGOS

Visa: fraude en un milisegundo

DATO
Cada transacción se evalúa en torno a 1 ms contra más de 500 atributos de riesgo.

DECISIÓN
El banco la aprueba, la rechaza o la marca.

RESULTADO
≈ 25.000 M$ de fraude evitado al año (estimación de Visa, 12 meses hasta abril de 2019).

INDUSTRIA

Rolls-Royce: motores que avisan

DATO
Los motores envían datos de rendimiento durante el vuelo.

DECISIÓN
Cuándo revisar un motor antes de que falle.

RESULTADO
Menos paradas imprevistas. La fuente describe el beneficio sin dar cifras.

DEPORTE

Fórmula 1: estrategia en directo

DATO
Más de 1,1 M de datos por segundo desde 300 sensores.

DECISIÓN
Cuándo parar en boxes y a qué rival atacar.

RESULTADO
Decisiones que se ajustan en la propia carrera, no después.

¿Qué decisión de tu sector perdería valor si el dato llegase un día tarde?

1.2 · VARIEDAD

Scouting: cuatro tipos de dato a la vez

ESTRUCTURADO

Tablas

Contratos, minutos jugados y lesiones.

`SQL · pandas`

SEMIESTRUCTURADO

Eventos JSON

Cada pase y cada tiro del partido, con su posición.

`Nuestro dataset`

NO ESTRUCTURADO

Texto

Informes de ojeadores y noticias de prensa.

`Lenguaje natural`

NO ESTRUCTURADO

Vídeo

La retransmisión de cada partido.

`Visión artificial`

Para valorar un fichaje hay que **cruzar los cuatro tipos**, y cada uno necesita su propia herramienta.

Spotify: tres tipos de dato, tres modelos

TABLA

Comportamiento

Qué escuchas, qué guardas y a qué artista vas después. Unos 140 M de usuarios y 30 M de canciones.

Filtrado colaborativo
Compara tu historial con el de otras personas.

TEXTO

Blogs y prensa

Artículos y metadatos sobre artistas y canciones.

Lenguaje natural
Términos clave con peso, que cambian cada día.

AUDIO

La pista en sí

El archivo de sonido sin procesar.

Redes neuronales
Recomiendan canciones nuevas con pocas escuchas.

Discover Weekly 30 canciones nuevas cada semana. Las listas personalizadas y editoriales pasaron de menos del 20 % de la escucha en 2015 a más del 30 % en 2017 (Stifel Research).

1.2 · VERACIDAD

Si el dato falla, falla la decisión

CASO REAL · 1999

Mars Climate Orbiter

Un equipo entregó el empuje en libras-fuerza y otro lo esperaba en newtons. La sonda se perdió al llegar a Marte.

≈ 125 M$

NUESTRO DATASET

Yardas, no metros

StatsBomb mide el campo en yardas: 120 × 80. Si las tratamos como metros, cada distancia sale un 9 % más larga.

120 yd ≈ 110 m

La unidad también es un dato: **hay que documentarla y validarla** antes de calcular.

Google Flu Trends: cuando el dato engaña

SALUD · GOOGLE FLU TRENDS

1. Google estima la gripe en directo a partir de las búsquedas de la gente.
2. Sobreestima la gripe en más del 50 % en 2011–12 y 2012–13, y se pasa en 100 de 108 semanas.
3. Cambian los servicios y los usuarios, y los datos de redes y buscadores se pueden manipular.

LO QUE FUNCIONÓ

Combinar big data con métodos tradicionales.

Los autores proponen una «revolución de todos los datos», no solo de los grandes.

LECCIÓN Más datos no son mejores datos: hay que validar de dónde vienen y si siguen midiendo lo mismo.

Oakland 2002: ganar sin dinero

PRESUPUESTO DE PLANTILLA EN 2002



Menos de un tercio. Y los jugadores estrella se marchaban a equipos con más dinero.

BASADO EN HECHOS REALES

Libro de Michael Lewis (2003) y película de 2011 con Brad Pitt como Billy Beane, director general de los Athletics.

Su ayudante: Paul DePodesta, en la película «Peter Brand».

¿Cómo compites con **tres veces menos** dinero?

Mismo béisbol, otra métrica

EL MERCADO PAGABA POR

- Promedio de bateo
- Home runs
- Carreras impulsadas

→

OAKLAND BUSCABA

- Porcentaje de embase (OBP)
- Slugging
- Bases por bolas

.374

Porcentaje de embase de **Scott Hatteberg** en 2002. Los A's lo reconvirtieron de receptor a primera base porque sabía llegar a base.

Comprar victorias, no jugadores

Llegar a base
barato y poco valorado

→

Más carreras
porque hay más corredores

→

Más victorias
lo que de verdad cuenta

«Tu objetivo no debería ser comprar jugadores, sino comprar victorias.»

Peter Brand en *Moneyball* (2011) · traducción libre

Resultado, matices y puente

103–59
balance de la temporada 2002

20
victorias seguidas, récord de la Liga Americana

3–2
derrota en la primera ronda ante Minnesota

Matiz: Oakland también tenía buenos lanzadores (Barry Zito ganó el Cy Young de 2002). Los datos ayudan, pero no explican todo.

Y EN FÚTBOL

Goles y asistencias son nuestro «promedio de bateo». ¿Qué cifra esconde el valor real de un jugador?

EJERCICIO · 3 MINUTOS

¿Quién tuvo el mejor partido?

1. En la tabla de la siguiente slide verás tres actuaciones de La Liga 2015/16, con cuatro cifras cada una.
2. En parejas, ordénalas de mejor a peor partido y apunta una razón.
3. Decide qué dato te faltaría para estar seguro.

Votamos a mano alzada y después destapamos el partido completo.

1.2 · VALOR (1 DE 2)

Tres jugadores, cuatro cifras

PARTIDO	ASIST.	GOLES	PASES OK	CON BALÓN
Las Palmas 1-1 Sporting	1	0	70 %	35 m
Celta 4-1 Barcelona	1	0	73 %	316 m
Sporting 0-3 Levante	1	1	78 %	180 m

Según estas cifras, el tercer jugador rinde el doble que el segundo y el primero lo iguala.

Ahora, el partido completo

PORTERO

Iván Cuéllar

Empate 1-1 en Las Palmas

- Pase largo de 44 m a gol
- 6 tiros enfrentados
- 3 paradas · 1 gol encajado
- Una salida fuera del área

DELANTERO

Lionel Messi

Derrota 4-1 en Vigo

- 6 ocasiones creadas
- 7 tiros · xG 0,58
- 9 de 11 regates ganados
- Un tiro al palo

CENTRAL

Zouhair Feddal

Levante ganó 0-3 en Gijón

- Gol con xG 0,94
- Asistencia desde 5 m
- 10 despejes · 3 bloqueos
- Dos veces regateado

El valor no está en la cifra, sino en su contexto.

Un solo ordenador no llega

MEMORIA

No cabe

Los datos no caben en la memoria, y a veces ni en el disco, de un solo equipo.

TIEMPO

Tarda demasiado

Procesar terabytes en una máquina puede llevar horas o días.

FALLOS

Algo siempre falla

Con muchas máquinas siempre hay un disco o un nodo que cae. El trabajo no puede perderse.

COSTE

Escala mal

Un servidor gigante cuesta mucho más que varios pequeños, y aun así tiene techo.

La salida: repartir el trabajo entre muchas máquinas.

Más máquinas, no máquinas más grandes

1 servidor

Scale up

Más RAM y CPU en la misma máquina. Caro y con techo.

Scale out

Muchas máquinas normales juntas. Crece casi sin límite.

La nube: máquinas por horas

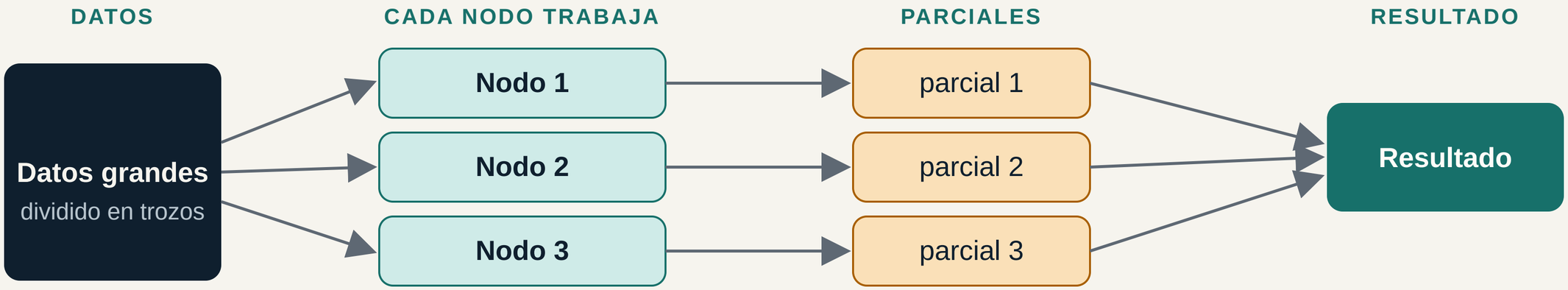
AWS, Azure y Google Cloud alquilan servidores. Pagas por uso y creces o encoges cuando quieras.

S3 · BigQuery · Databricks

EN EL CURSO

Colab y Databricks gratis, con recursos limitados. Por eso trabajamos con datos pequeños.

Dividir, calcular, combinar



Coordinar

Quién hace qué y quién junta los resultados.

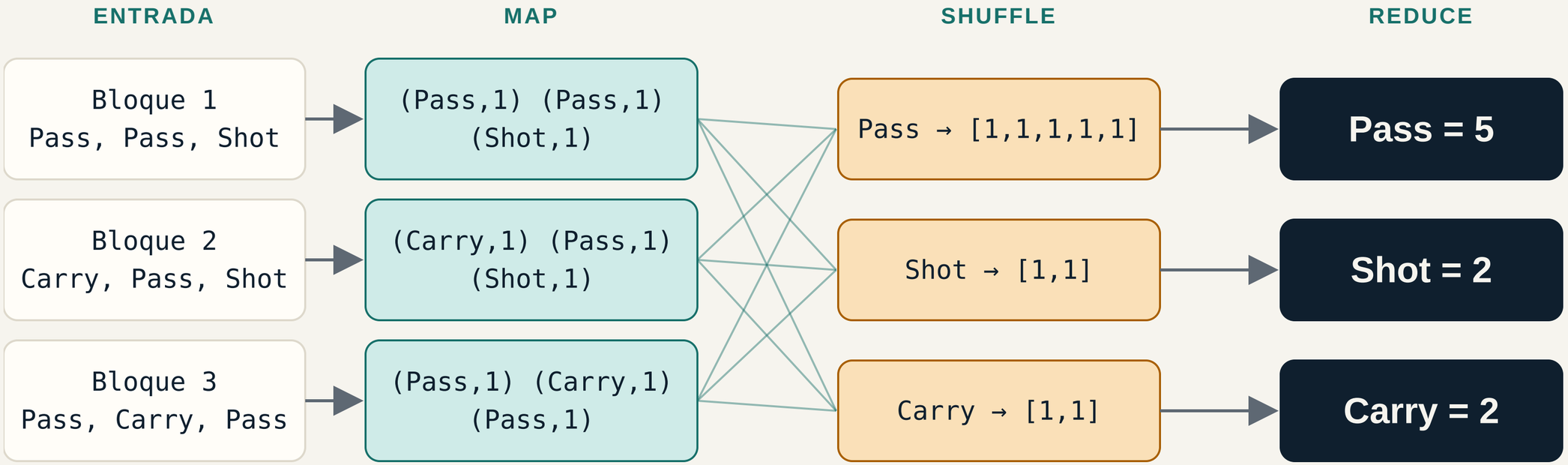
Tolerar fallos

Si un nodo cae, su trozo se repite en otro.

Mover poco

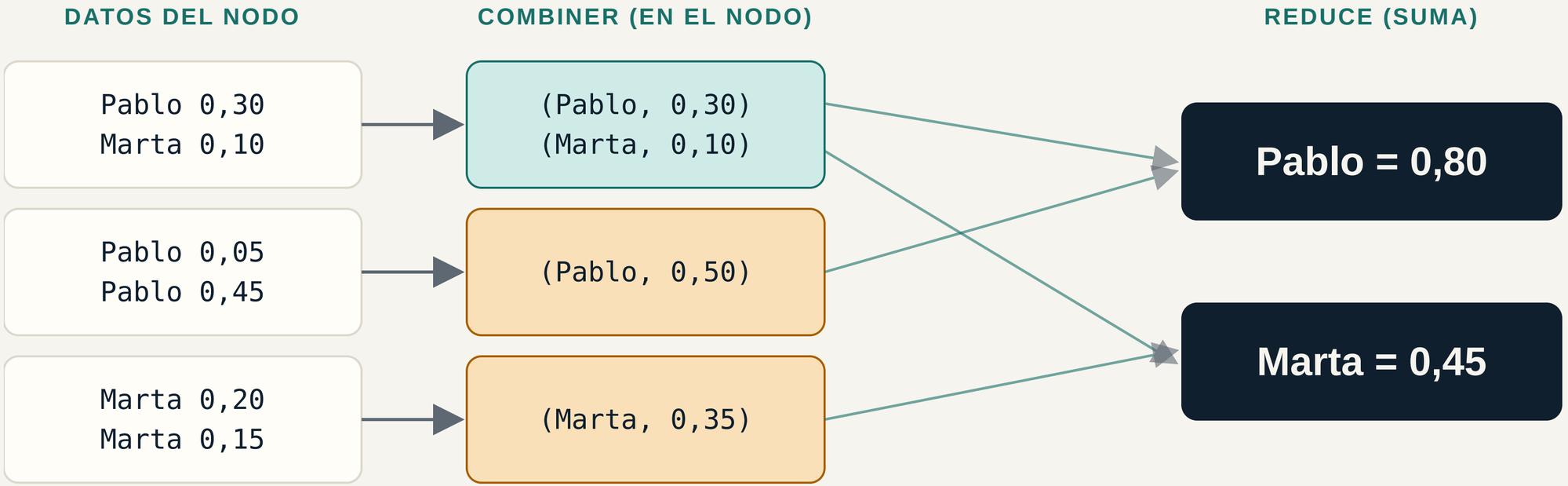
Llevar el cálculo a los datos, no los datos por la red.

Contar eventos con MapReduce



Map emite (clave, 1) · **Shuffle** agrupa por clave · **Reduce** suma. Datos de ejemplo, no reales.

Sumar xG por jugador y ahorrar red



Combiner: el nodo suma antes de enviar, así viajan 4 pares en vez de 6. Vale porque la suma es asociativa. Datos inventados.

El mismo ejemplo en código

python · en un solo ordenador

```python
bloques = [["Pass", "Pass", "Shot"],
           ["Carry", "Pass", "Shot"],
           ["Pass", "Carry", "Pass"]]
# MAP: cada bloque emite (clave, 1)
mapa = [(t, 1) for b in bloques for t in b]
# SHUFFLE: agrupar valores por clave
grupos = {}
for k, v in mapa:
   grupos.setdefault(k, []).append(v)
# REDUCE: sumar los valores de cada clave
final = {k: sum(v) for k, v in grupos.items()}
# {'Pass': 5, 'Shot': 2, 'Carry': 2}
```

pyspark · en un clúster

```python
# Spark: lo mismo, repartido en el clúster
(sc.parallelize(bloques, 3)
  .flatMap(lambda b: b)
  .map(lambda t: (t, 1))
  .reduceByKey(lambda a, b: a + b)
  .collect())
# o con DataFrames:
df.groupBy("type").count().show()
```

Spark hace lo mismo, pero **en memoria** y repartido por el clúster.

Datos estructurados y no estructurados

Tipo	Ejemplos	Herramientas típicas
Estructurados	Tablas SQL, CSV, hojas de cálculo	SQL, pandas
Semiestructurados	JSON, XML, registros de servidores	Spark, MongoDB
No estructurados	Texto, imágenes, audio, vídeo	Data lakes, deep learning

Nuestro caso práctico parte de datos **semiestructurados**: JSON de eventos de fútbol que convertiremos en tablas.

Cada tipo de dato, su almacén

ESTRUCTURADOS

Bases de datos relacionales

Tablas con filas y columnas y un esquema fijo.

`MySQL · PostgreSQL`

SEMIESTRUCTURADOS

Bases de datos documentales

Documentos JSON cuyos campos pueden variar.

`MongoDB · Firestore`

NO ESTRUCTURADOS

Data lakes y almacenes de objetos

Ficheros guardados tal cual, sin esquema previo.

`Amazon S3 · HDFS`

Veremos un ejemplo de cada uno: **MySQL, MongoDB y un data lake**.

MySQL: tablas relacionadas

- Datos en tablas de filas y columnas
- Esquema fijo: los tipos se definen antes
- Clave primaria **id** en cada tabla
- Clave foránea **id_cliente** que las relaciona
- Se consulta con SQL

clientes

id	nombre	ciudad
1	Ana López	Madrid
2	Luis Pérez	Sevilla
3	Marta Ruiz	Valencia

contratos

id_contrato	id_cliente	tipo	cuota_mensual
101	1	Mensual	70.35
102	2	Un año	89.10
103	3	Dos años	45.20

Consultar con SQL

mysql · consulta

```
SELECT c.nombre, k.tipo, k.cuota_mensual
FROM clientes c
JOIN contratos k ON k.id_cliente = c.id
WHERE k.cuota_mensual > 60;
```

RESULTADO

nombre	tipo	cuota_mensual
Ana López	Mensual	70.35
Luis Pérez	Un año	89.10

El **JOIN** une las tablas por su clave: cada dato vive en un solo sitio.

Un JOIN no es gratis

clientes
1 · Ana
2 · Luis
3 · Marta



contratos
101 → 1
102 → 2
103 → 3

Para cada fila de una tabla, el motor **busca su pareja** en la otra. Con pocas filas es instantáneo; con millones, es trabajo.

Bucle anidado

Compara cada fila con todas las de la otra tabla. Con índice, busca directamente; sin él, el coste se multiplica.

Hash join

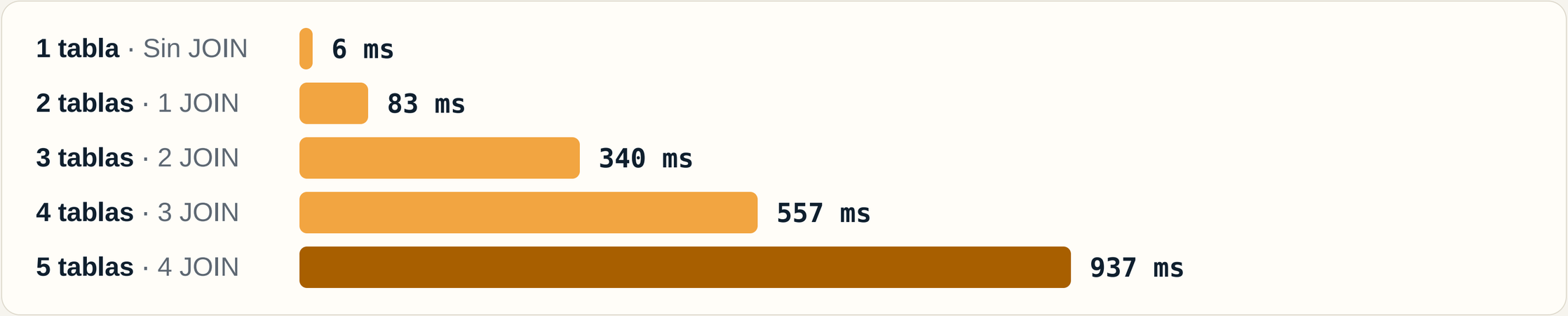
Construye una tabla hash con una de las dos y consulta con la otra. Rápido, pero gasta memoria.

Merge join

Ordena las dos por la clave y las recorre a la vez. Va bien si ya están ordenadas.

El coste depende de cuántas filas hay, de si hay índices y de cuántos datos intermedios se generan.

Cuantas más tablas, más tarda



Medición propia con SQLite en memoria: 200.000 filas por tabla, claves con índice, datos sintéticos, mediana de 5 ejecuciones. Las cifras varían según la máquina y el motor; importa la tendencia.

De 1 a 5 tablas, la misma consulta pasa de milisegundos a casi un segundo: unas 150 veces más.

Cómo se mantiene el coste bajo control

Normalizar crea más tablas; estas son las herramientas para que leer no se encarezca.

Índices en las claves

Buscar por clave deja de recorrer la tabla entera.

Filtrar antes de unir

Un WHERE que reduce filas abarata todos los JOIN posteriores.

Pedir solo lo necesario

Menos columnas y menos filas viajan entre operaciones.

Mirar el plan

EXPLAIN enseña cómo ejecutará el motor la consulta.

Desnormalizar al leer

Repetir datos a propósito cuando se lee mucho y se escribe poco.

Modelo en estrella

Pocas dimensiones y un solo nivel de JOIN para analítica.

Ahora, **las formas normales**: cómo separar bien los datos sin perder de vista este coste.

Una tabla para todo

pedidos · una tienda de fútbol

id_pedido	id_cliente	cliente	ciudad	productos
1	1	Ana López	Madrid	Camiseta x2 (25.00), Balón x1 (30.00)
2	2	Luis Pérez	Sevilla	Balón x1 (30.00)
3	1	Ana López	Madrid	Camiseta x1 (25.00)

Datos repetidos

Ana López y Madrid aparecen en dos pedidos.

Cambios arriesgados

Si Ana se muda, hay que tocar varias filas.

Borrados que pierden datos

Borrar el pedido 2 elimina también a Luis Pérez.

1FN: un valor por celda

Antes · la celda productos guarda una lista

id_pedido	id_cliente	cliente	ciudad	productos
1	1	Ana López	Madrid	Camiseta x2 (25.00), Balón x1 (30.00)
2	2	Luis Pérez	Sevilla	Balón x1 (30.00)
3	1	Ana López	Madrid	Camiseta x1 (25.00)

↓

Después · una fila por producto y pedido

id_pedido	id_cliente	cliente	ciudad	id_producto	producto	precio	cantidad
1	1	Ana López	Madrid	P1	Camiseta	25.00	2
1	1	Ana López	Madrid	P2	Balón	30.00	1
2	2	Luis Pérez	Sevilla	P2	Balón	30.00	1
3	1	Ana López	Madrid	P1	Camiseta	25.00	1

Una celda, un valor. La clave pasa a ser **(id_pedido, id_producto)**.

EJERCICIO · 5 MINUTOS

1FN: los jugadores de la cantera

Un club de cantera guarda sus jugadores así:

jugadores

id_jugador	nombre	equipo	posiciones
1	Pablo Pérez	Gijón Norte	Delantero, Extremo
2	Marta Gil	Gijón Norte	Portera
3	Iván Sol	Avilés Sur	Central, Lateral

1. ¿Qué columna rompe la 1FN y por qué?
2. Reescribe la tabla para que la cumpla.
3. ¿Cuál es ahora la clave?

SOLUCIÓN · 1FN

Una fila por posición

jugadores

id_jugador	nombre	equipo	posicion
1	Pablo Pérez	Gijón Norte	Delantero
1	Pablo Pérez	Gijón Norte	Extremo
2	Marta Gil	Gijón Norte	Portera
3	Iván Sol	Avilés Sur	Central
3	Iván Sol	Avilés Sur	Lateral

Qué rompía la regla

La columna posiciones guardaba una lista en una sola celda.

La clave

Ahora es la pareja (id_jugador, posicion).

2FN: nada depende de media clave

Antes · clave (id_pedido, id_producto)

id_pedido	id_cliente	cliente	ciudad	id_producto	producto	precio	cantidad
1	1	Ana López	Madrid	P1	Camiseta	25.00	2
1	1	Ana López	Madrid	P2	Balón	30.00	1
2	2	Luis Pérez	Sevilla	P2	Balón	30.00	1
3	1	Ana López	Madrid	P1	Camiseta	25.00	1

Naranja depende solo de id_pedido · Verde depende solo de id_producto · Solo cantidad necesita las dos.

pedidos

id_pedido	id_cliente	cliente	ciudad
1	1	Ana L.	Madrid
2	2	Luis P.	Sevilla
3	1	Ana L.	Madrid

lineas_pedido

id_pedido	id_producto	cantidad
1	P1	2
1	P2	1
2	P2	1
3	P1	1

productos

id_producto	producto	precio
P1	Camiseta	25.00
P2	Balón	30.00

EJERCICIO · 5 MINUTOS

2FN: las estadísticas de la liga

Esta tabla cumple 1FN y su clave es **(id_partido, id_jugador)**:

estadisticas

id_partido	id_jugador	fecha	rival	nombre_jugador	goles
10	1	2026-09-12	Costa FC	Pablo Pérez	2
10	3	2026-09-12	Costa FC	Iván Sol	0
11	1	2026-09-19	Mar FC	Pablo Pérez	1

1. ¿Qué columnas dependen solo de id_partido?
2. ¿Cuáles dependen solo de id_jugador?
3. Reparte las columnas en tablas que cumplan 2FN.

SOLUCIÓN · 2FN

Tres tablas, cada una con su clave

partidos

id_partido	fecha	rival
10	2026-09-12	Costa FC
11	2026-09-19	Mar FC

jugadores

id_jugador	nombre
1	Pablo Pérez
3	Iván Sol

estadisticas

id_partido	id_jugador	goles
10	1	2
10	3	0
11	1	1

Dependen de una parte

fecha y rival, solo de id_partido. nombre, solo de id_jugador.

Dependen de la clave entera

goles necesita partido y jugador: se queda en estadisticas.

3FN: sin dependencias ocultas

Antes · pedidos

id_pedido	id_cliente	cliente	ciudad
1	1	Ana López	Madrid
2	2	Luis Pérez	Sevilla
3	1	Ana López	Madrid



pedidos

id_pedido	id_cliente
1	1
2	2
3	1

clientes

id_cliente	nombre	ciudad
1	Ana López	Madrid
2	Luis Pérez	Sevilla

El problema

La ciudad no depende del pedido, sino del cliente: id_pedido determina id_cliente, y éste determina la ciudad.

La solución

Una tabla clientes. Ana López y Madrid ya aparecen una sola vez.

EJERCICIO · 5 MINUTOS

3FN: equipos y estadios

Esta tabla cumple 1FN y 2FN. Su clave es **id_jugador**:

jugadores

id_jugador	nombre	id_equipo	equipo	estadio
1	Pablo Pérez	1	Gijón Norte	Estadio Norte
2	Marta Gil	1	Gijón Norte	Estadio Norte
3	Iván Sol	2	Avilés Sur	Estadio Sur

1. ¿Qué columnas dependen de id_equipo y no del jugador?
2. ¿Qué pasa si el equipo cambia de estadio?
3. Divide la tabla para que cumpla 3FN.

SOLUCIÓN · 3FN

Los datos del equipo, en su tabla

jugadores

id_jugador	nombre	id_equipo
1	Pablo Pérez	1
2	Marta Gil	1
3	Iván Sol	2

equipos

id_equipo	equipo	estadio
1	Gijón Norte	Estadio Norte
2	Avilés Sur	Estadio Sur

La dependencia oculta

equipo y estadio dependen de id_equipo, no del jugador.

Qué se gana

Si cambia el estadio, se modifica una sola fila de equipos.

Resultado: cuatro tablas relacionadas

clientes
id_cliente PK
nombre
ciudad

pedidos
id_pedido PK
id_cliente FK

lineas_pedido
id_pedido PK, FK
id_producto PK, FK
cantidad

productos
id_producto PK
producto
precio

mysql · recomponer la tabla original

```
SELECT c.nombre, c.ciudad, p.producto, l.cantidad
FROM pedidos o
JOIN clientes c ON c.id_cliente = o.id_cliente
JOIN lineas_pedido l ON l.id_pedido = o.id_pedido
JOIN productos p ON p.id_producto = l.id_producto;
```

Cada dato vive en un solo sitio.

Los JOIN devuelven la información original cuando hace falta.

¿Normalizar siempre?

POR QUÉ SÍ

- Cada dato vive en un solo sitio
- Un cambio se hace una vez
- Menos espacio y menos errores
- Las claves garantizan la integridad

POR QUÉ NO

- Más JOINs y consultas más complejas
- Lecturas lentas con mucho volumen
- Más tablas que mantener y entender
- En analítica se repite el dato a propósito

Para transacciones, normalizar. Para analítica, tablas anchas, como nuestro CSV de eventos.

Don't forget the WHERE

mysql · consola

```
mysql> DELETE FROM clientes;
Query OK, 3 rows affected (0.01 sec)

-- Lo que querías hacer:
mysql> DELETE FROM clientes WHERE id = 2;
Query OK, 1 row affected (0.01 sec)
```

VIERNES, 17:59

`DELETE FROM clientes;`

ADIÓS, TODAS LAS FILAS

1 · SELECT primero

Ejecuta el mismo WHERE con SELECT y revisa las filas.

2 · Transacción

START TRANSACTION; y, si algo va mal, ROLLBACK;

3 · Modo seguro

SET sql_safe_updates = 1; bloquea borrados sin filtro por clave.

MongoDB: documentos flexibles

Relacional	MongoDB
Tabla	**Colección**
Fila	**Documento**
Columna	**Campo**
JOIN entre tablas	**Datos anidados**

- Cada documento es un objeto en formato JSON
- Dos documentos de la misma colección pueden tener campos distintos
- No hay esquema fijo: se añaden campos cuando hacen falta
- Encaja con datos anidados y cambiantes

Documentos y consultas

clientes · documento

```
{
  "_id": 2,
  "nombre": "Luis Pérez",
  "contrato": {
    "tipo": "Un año",
    "cuota_mensual": 89.10
  },
  "servicios": ["internet", "tv"]
}
```

mongosh · consulta

```
db.clientes.find(
  { "contrato.cuota_mensual": { $gt: 60 } },
  { nombre: 1, "contrato.tipo": 1 }
)

[
  { _id: 1, nombre: 'Ana López', contrato: { tipo: 'Mensual' } },
  { _id: 2, nombre: 'Luis Pérez', contrato: { tipo: 'Un año' } }
]
```

Data lakes: guardar primero

Texto

Correos, PDF, opiniones.

Imágenes

Fotos, escaneos, radiografías.

Audio y vídeo

Llamadas, grabaciones, clips.

Logs y sensores

Registros continuos de máquinas.

ESQUEMA AL LEER

Se guarda primero y se interpreta después, según el uso.

ALMACENES HABITUALES

`Amazon S3 · Azure Data Lake · HDFS`

Dentro de un data lake

CONTENIDO DE UN ALMACÉN DE OBJETOS

Ruta	Tipo	Tamaño
raw/imagenes/tienda_001.jpg	Imagen	240 KB
raw/documentos/contrato_017.pdf	Documento	1,2 MB
raw/audio/llamada_0412.mp3	Audio	3,8 MB
raw/logs/app_2026-11-03.log	Texto	15 MB



Comparativa de los tres tipos

Criterio	Relacional (MySQL)	Documental (MongoDB)	Data lake (S3, HDFS)
Datos	Tablas con filas	Documentos JSON	Ficheros de cualquier tipo
Esquema	Fijo, antes de guardar	Flexible	Se decide al leer
Consulta	SQL	Consultas por documento	Spark u otros motores
Encaja con	Datos tabulares y transacciones	Datos anidados y cambiantes	Grandes volúmenes en bruto

No se excluyen: un proyecto real suele **combinar los tres**.

Redis: velocidad ante todo

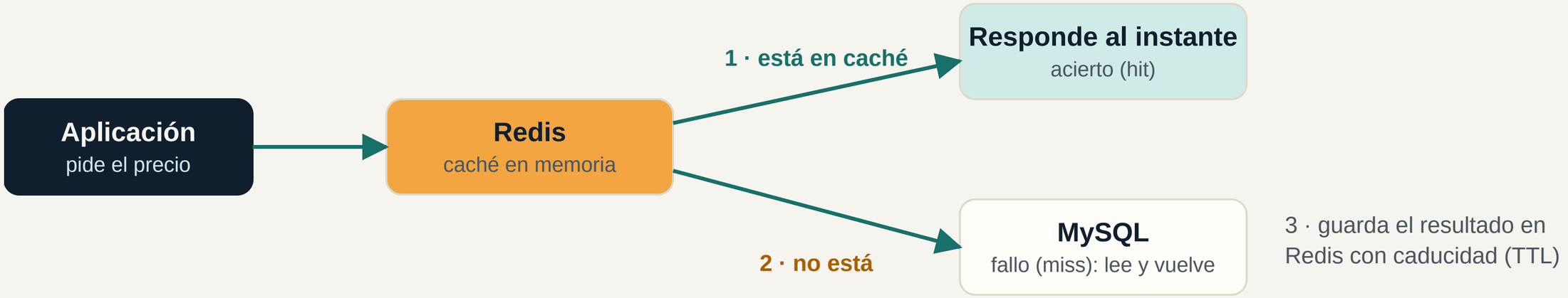
- Estructura mínima: una clave y su valor
- Trabaja en memoria, con respuestas muy rápidas
- Los valores pueden caducar pasado un tiempo
- No sirve para consultas complejas

Caché **Sesiones** **Contadores** **Rankings**

redis · consola

```
redis> SET sesion:ana "activa" EX 60
OK
redis> GET sesion:ana
"activa"
redis> INCR visitas:portada
(integer) 1
```

Redis delante de la base de datos



QUÉ SE GANA

Las consultas repetidas se responden desde memoria y MySQL se libera de carga: lo que más se pide, más rápido.

QUÉ HAY QUE CUIDAR

Un dato en caché puede quedarse viejo: hay que darle caducidad o borrarlo cuando cambia en MySQL.

La caché en pocas líneas de Python

python · cache-aside (ilustrativo)

```python
import json, redis
r = redis.Redis()
def precio(sku):
    hit = r.get(f"precio:{sku}")
    if hit: return json.loads(hit) # acierto: sin tocar MySQL
    fila = consultar_mysql(sku) # fallo: a la base de datos
    r.set(f"precio:{sku}", json.dumps(fila), ex=300) # caduca en 5 min
    return fila
```

Acierto

Redis tiene la clave: MySQL ni se entera.

Fallo

Se consulta MySQL y se guarda con ex=300.

Si el dato cambia

Borrar la clave con r.delete().

Neo4j: la base de datos de grafos

Cómo piensa Neo4j

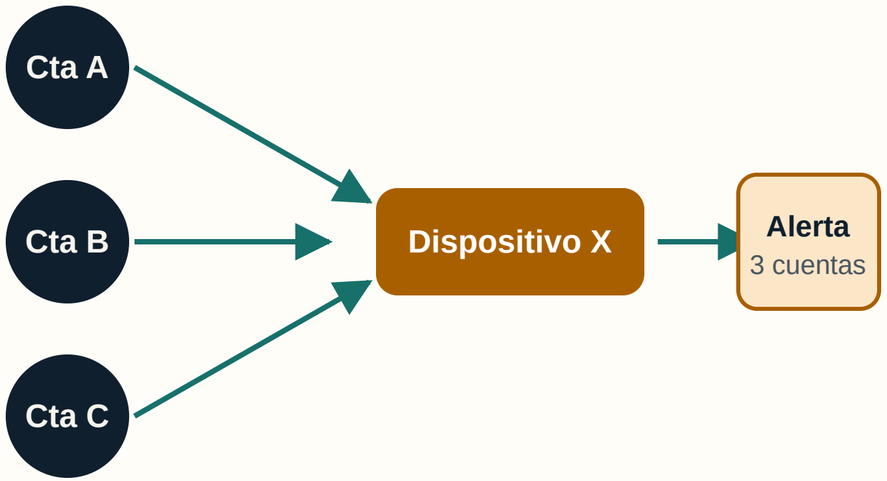
- Nodos: personas, productos, cuentas
- Relaciones con nombre y dirección: SIGUE, COMPRO
- Se consulta con Cypher: dibujas el patrón que buscas
- Recorrer relaciones no necesita JOIN

neo4j · Cypher

```
MATCH (ana:Persona {nombre: "Ana"})-[:SIGUE]->(amigo)
      -[:COMPRO]->(p:Producto)
WHERE NOT (ana)-[:COMPRO]->(p)
RETURN p.nombre, count(*) AS amigos
ORDER BY amigos DESC
```

«Productos que compraron las personas a las que sigue Ana y que ella aún no tiene.»

Caso de uso: detectar fraude en un banco



neo4j · Cypher

```
MATCH (a:Cuenta)-[:USA]->(d:Dispositivo)
      <-[:USA]-(b:Cuenta)
WHERE a <> b
RETURN d.id, collect(DISTINCT a.id) AS cuentas
```

Todas las flechas son la relación **USA**. La consulta devuelve cada dispositivo compartido y las cuentas que lo usan.

Fuerte: ve redes enteras. **Débil:** hay que modelar bien nodos y relaciones.

Una cuenta sola parece normal; tres cuentas con el mismo dispositivo ya es una pista.

Seis formas de organizar el dato

Cada modelo responde bien a un tipo de pregunta. Esto es lo que veremos de cada uno:

Relacional

Tablas con claves que las relacionan.

Pregunta: ¿qué pedidos hizo cada cliente?

Estrella

Una tabla de hechos rodeada de dimensiones.

Pregunta: ¿cuánto vendimos por mes y tienda?

Documental

Documentos JSON anidados y flexibles.

Pregunta: ¿cómo es la ficha completa de este producto?

Clave-valor

Una clave y su valor, nada más.

Pregunta: ¿cuál es el valor de esta clave, ya?

Columnar

Los datos se guardan por columnas.

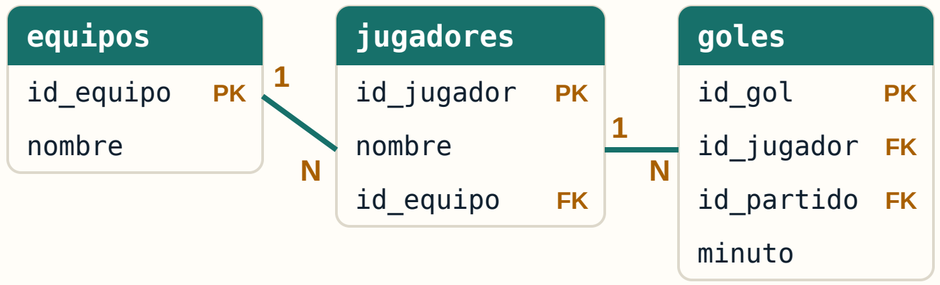
Pregunta: ¿cuál es la media de este campo en 1.000 millones de filas?

Grafo

Nodos y relaciones entre ellos.

Pregunta: ¿quién está conectado con quién, y a cuántos saltos?

Relacional: tablas y claves



Un equipo tiene muchos jugadores;
un jugador, muchos goles.

ENCAJA CON Pedidos, pagos, inventario: datos tabulares con reglas.

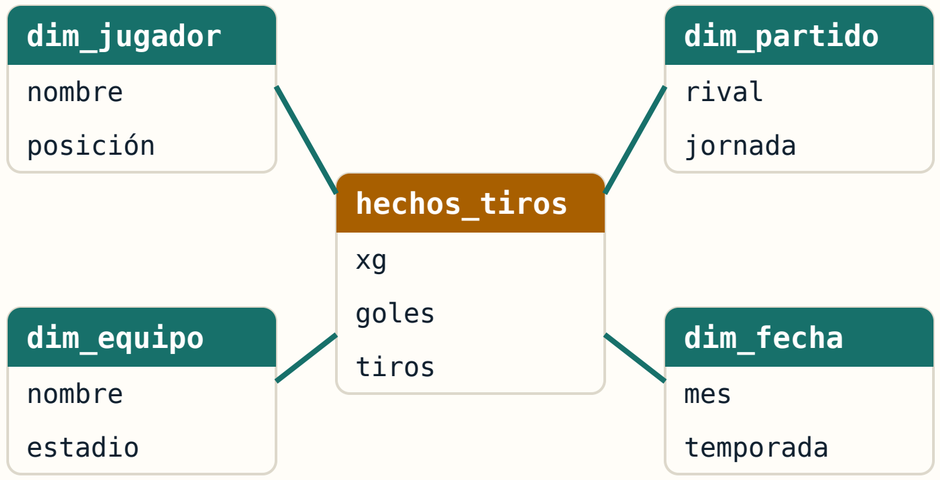
PUNTOS FUERTES

- Integridad: claves y reglas evitan datos incoherentes
- SQL estándar y joins entre tablas
- Transacciones ACID: o se guarda todo o nada
- Muy maduro: herramientas y personas formadas

PUNTOS DÉBILES

- Esquema rígido: cambiarlo cuesta
- Muchos joins pesan a gran escala
- Escalar a varias máquinas es más difícil

Estrella: hechos y dimensiones



ENCAJA CON Almacenes de datos (data warehouse) y dashboards.

PUNTOS FUERTES

- Consultas analíticas simples: pocos joins
- Fácil de entender para negocio y BI
- Agrega muy bien: por jugador, mes o rival

PUNTOS DÉBILES

- Datos repetidos en las dimensiones
- Cambiar el esquema obliga a rehacer cargas
- No sirve para operaciones transaccionales

Documental: el objeto completo

documento.json

```
{
  "jugador": "Ana Ruiz",
  "equipo": "Sporting",
  "temporadas": [
    { "año": 2015, "goles": 7 },
    { "año": 2016, "goles": 4 }
  ],
  "lesiones": []
}
```

ENCAJA CON Catálogos, perfiles de usuario, datos que cambian de forma.

PUNTOS FUERTES

- Esquema flexible: cada documento puede ser distinto
- Guarda el objeto entero, con datos anidados
- Escala repartiendo documentos entre máquinas

PUNTOS DÉBILES

- Las relaciones entre documentos son débiles
- Se duplican datos y cuesta mantenerlos igual
- Transacciones entre documentos más limitadas

Clave-valor: acceso directo

`sesion:ana`	→ `"activa"`	caduca 60 s
`visitas:portada`	→ `1532`	
`precio:sku42`	→ `19.90`	
`ranking:goles`	→ `["Ana","Luis"]`	

ENCAJA CON Cachés, sesiones, contadores y rankings (Redis, DynamoDB).

PUNTOS FUERTES

- Latencia mínima: buscar por clave es inmediato
- Muy simple y fácil de escalar
- Caducidad automática de valores

PUNTOS DÉBILES

- Solo se busca por clave, no por contenido
- Sin consultas complejas ni relaciones
- En memoria, el GB es caro

Columnar: leer solo lo necesario

Por filas

1	Ana	20	2	Luis	35	3	Marta	15

Por columnas

1	2	3	Ana	Luis	Marta	20	35	15

SUM(importe) lee solo las celdas naranjas:
no necesita leer id ni nombre.

Mismos datos; cambia el orden en disco.

ENCAJA CON Análisis masivo: Parquet, BigQuery, ClickHouse, warehouses.

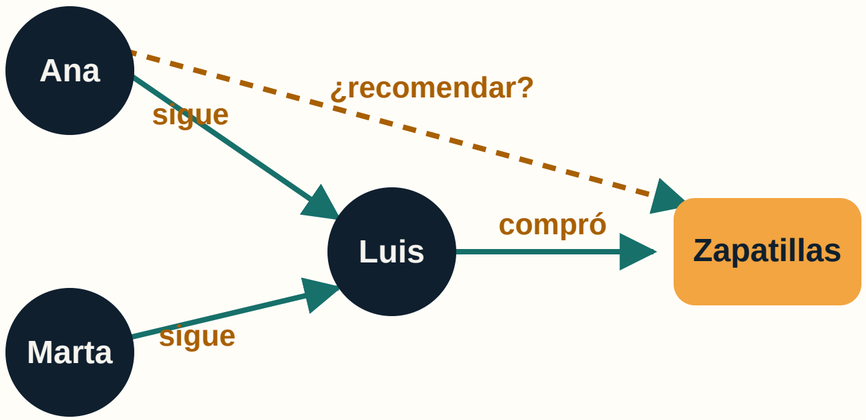
PUNTOS FUERTES

- Lee solo las columnas que pide la consulta
- Comprime muy bien: valores parecidos juntos
- Rápido en sumas y medias sobre millones de filas

PUNTOS DÉBILES

- Lento para cambiar o insertar filas sueltas
- Reconstruir una fila entera cuesta más
- No pensado para transacciones

Grafo: lo importante son las relaciones



ENCAJA CON Redes sociales, recomendaciones, fraude y rutas (Neo4j).

PUNTOS FUERTES

- Las relaciones son datos de primera clase
- Recorrer varios saltos es rápido
- Natural para recomendar, detectar fraude y redes

PUNTOS DÉBILES

- Mal para sumas y medias sobre toda la base
- Cuesta más repartirlo entre máquinas
- Lenguaje de consulta propio (Cypher, Gremlin)

1.5 · RESUMEN

Elegir el modelo por la pregunta

	MEJOR PARA	FLOJEA EN	EJEMPLOS
Relacional	Datos con reglas y transacciones	Escalar y cambiar el esquema	MySQL, PostgreSQL
Estrella	Análisis y dashboards	Operaciones y cambios frecuentes	Data warehouse
Documental	Objetos anidados y cambiantes	Relaciones entre documentos	MongoDB
Clave-valor	Respuesta inmediata por clave	Buscar por contenido	Redis, DynamoDB
Columnar	Agregar millones de filas	Escrituras de filas sueltas	Parquet, BigQuery
Grafo	Conexiones entre entidades	Agregaciones globales	Neo4j

La pregunta de negocio decide el modelo. Un proyecto real suele combinar varios.

Muchos modelos, dos problemas

Relacional · Estrella · Documental · Clave-valor · Columnar · Grafo · Data lake

1

Consistencia

¿Cómo lo mantengo coherente?

Si el mismo dato vive en varios sitios o en varias máquinas, ¿quién tiene la versión buena y cuándo se ve el cambio?

PRIMERO · TEOREMA CAP

2

Almacenamiento

¿Dónde guardo cada dato?

Memoria, disco, almacén de objetos o archivo: cada capa cuesta y responde distinto, y el volumen no deja de crecer.

DESPUÉS · CAPAS DE ALMACENAMIENTO

El teorema CAP

C · Consistencia

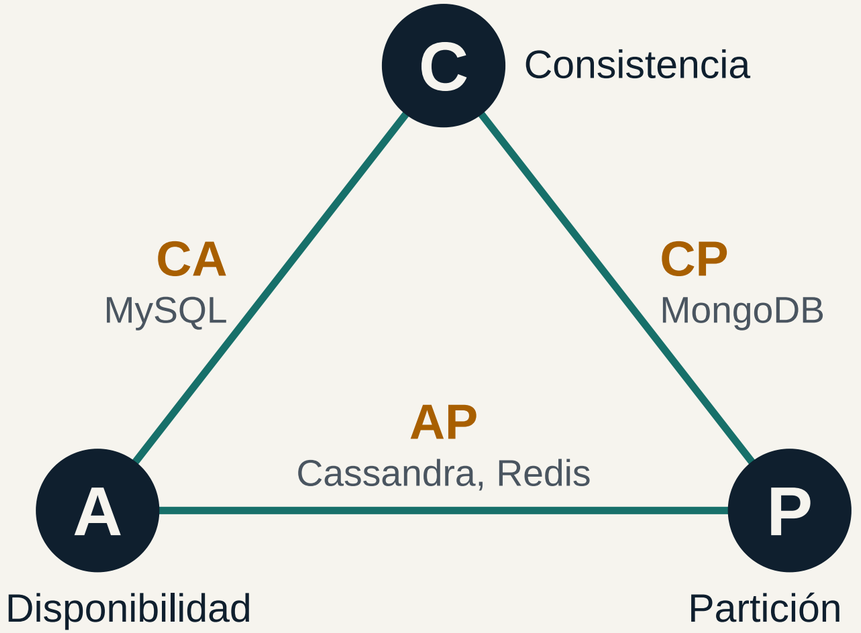
Todos ven el mismo dato a la vez.

A · Disponibilidad

Cada petición recibe una respuesta.

P · Tolerancia a particiones

Sigue funcionando si se corta la red.



Las particiones ocurren: el dilema real es **consistencia o disponibilidad**.

CAP: relacionales frente a NoSQL

SISTEMA	PRIORIZA	SI SE CORTA LA RED
MySQL (1 servidor)	CA	No hay red entre nodos: coherencia total
MongoDB	CP	Sin mayoría de nodos, no acepta escrituras
Redis (réplicas)	AP	Responde rápido, pero puede perder escrituras
Cassandra	AP	Siempre responde; los nodos se alinean después

ACID · relacional

Atomicidad, Consistencia, Aislamiento y Durabilidad.

BASE · muchos NoSQL

Disponible, estado flexible y consistencia eventual.

1.6 · ALMACENAMIENTO

Capas de almacenamiento: de la memoria al archivo

Más rápido arriba, más barato abajo: cada dato debe estar donde se use.

CAPA 1 **Memoria · caliente**	Redis y cachés Respuesta en milisegundos o menos · coste por GB muy alto
CAPA 2 **Disco rápido (SSD) · caliente**	MySQL, MongoDB: bases operacionales Milisegundos · coste por GB alto
CAPA 3 **Almacén de objetos · tibio**	S3, HDFS, data lake y warehouse Acceso inmediato; consultas por lotes · coste por GB bajo
CAPA 4 **Archivo · frío (cold storage)**	S3 Glacier, cintas De minutos a horas para recuperar · coste por GB muy bajo

1.6 · ALMACENAMIENTO

¿En qué capa va cada dato?

Ejemplo ilustrativo con un club de fútbol.

	CON QUÉ FRECUENCIA SE USA	CAPA
Marcador y alineación en directo	Constantemente	Capa 1 · memoria
Plantilla, fichajes y calendario de la temporada	Varias veces al día	Capa 2 · base de datos
Los 128.813 eventos de la temporada para análisis	Cada semana, por lotes	Capa 3 · lake / warehouse
Vídeos y datos brutos de hace varios años	Casi nunca	Capa 4 · archivo

Regla práctica: cuanto menos se usa, más abajo y más barato.

Cold storage: guardar barato lo que casi no se usa

Más barato por GB, a cambio de recuperar más despacio.

¿Por qué?

Obligaciones legales, auditorías, copias de seguridad e histórico para entrenar modelos en el futuro.

¿Cómo?

Reglas de ciclo de vida: tras X días, el dato pasa solo a una capa más barata, y a los N años se borra.

Trampas

Recuperar tarda y puede costar; hay plazos mínimos de permanencia; y el RGPD obliga a no conservar más de lo necesario.

Un dato en frío no es un dato olvidado: **se guarda con un motivo y una fecha de caducidad**.

El dato envejece: ciclo de vida y cold storage real

POLÍTICA DE CICLO DE VIDA (EJEMPLO ILUSTRATIVO)

DÍAS 0–30
Capa 2–3
Se consulta a diario

DÍAS 30–90
Capa 3
Se consulta de vez en cuando

DESDE EL DÍA 90
Capa 4 · archivo
Casi no se consulta

AÑO 7 (SUPUESTO)
Borrado
Termina el plazo de conservación

CLASES DE ARCHIVO EN AWS (SEGÚN SU DOCUMENTACIÓN)

	RECUPERACIÓN MEDIA	PERMANENCIA MÍNIMA
Glacier Instant Retrieval	Milisegundos	90 días
Glacier Flexible Retrieval	Minutos a 12 horas	90 días
Glacier Deep Archive	9 a 48 horas	180 días

Arquitectura: cómo fluye el dato

1
Entra
Eventos, ficheros, APIs, sensores.

2
Se guarda
Capas y modelos que ya conocemos.

3
Se transforma
Limpiar, unir y calcular.

4
Se sirve
Dashboards, apps y modelos.

TRES DECISIONES QUE LA MOLDEAN

¿Cuándo se necesita?
Horas, minutos o segundos: la latencia manda.

¿Cuánto dato hay?
Del portátil al clúster: el volumen cambia las herramientas.

¿Cuánta coherencia exige?
Un saldo bancario no admite errores; un contador de visitas sí.

1.7 · TIEMPO REAL

Lote, micro-lote y streaming

Una escala, no tres opciones separadas: cuánto tardamos en procesar lo que llega.

	CUÁNDO PROCESA	LATENCIA TÍPICA	EJEMPLO
Lote (batch)	Cuando se acumula un volumen o llega la hora	Horas o días	Informe mensual de ventas
Micro-lote	En pequeños lotes cada pocos segundos	Segundos a minutos	Dashboard que se refresca cada minuto
Streaming	Evento a evento, según llega	Milisegundos a segundos	Alerta de fraude al pagar

Cuanto más rápido responde, más complejo y caro es de operar.

Colas y mensajería: el cartero del dato



QUÉ APORTA

Desacopla a quien produce de quien consume, absorbe los picos de tráfico y permite reintentar si algo falla.

COLA FRENTE A LOG

En una cola el mensaje se entrega y desaparece (RabbitMQ, SQS). En un log se conserva y se puede releer (Kafka).

Entrega de mensajes: tres garantías

Con una red de por medio, los mensajes se pierden, se repiten o llegan desordenados.

A lo sumo una vez

Puede perderse algún mensaje, pero nunca llega duplicado.

Al menos una vez

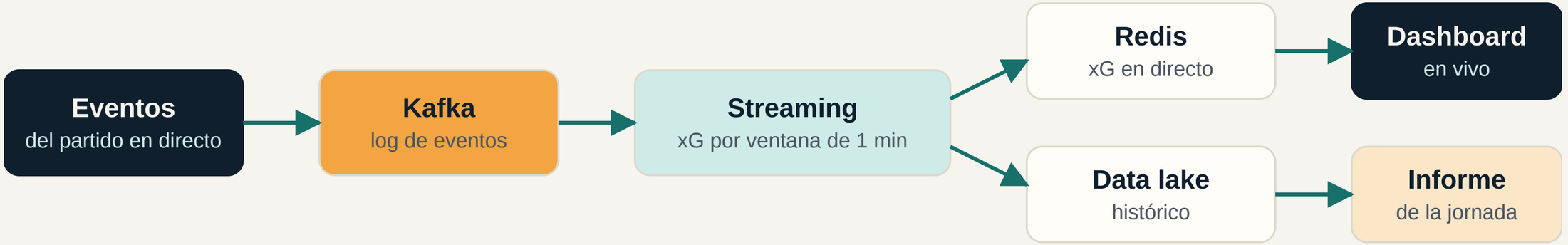
No se pierde ninguno, pero puede llegar repetido.

Exactamente una vez

Lo ideal: ni pérdidas ni duplicados. Es difícil y caro de conseguir.

Truco habitual: que el consumidor sea **idempotente**: procesar un mensaje dos veces da el mismo resultado que una.

Ejemplo: seguir un partido en directo



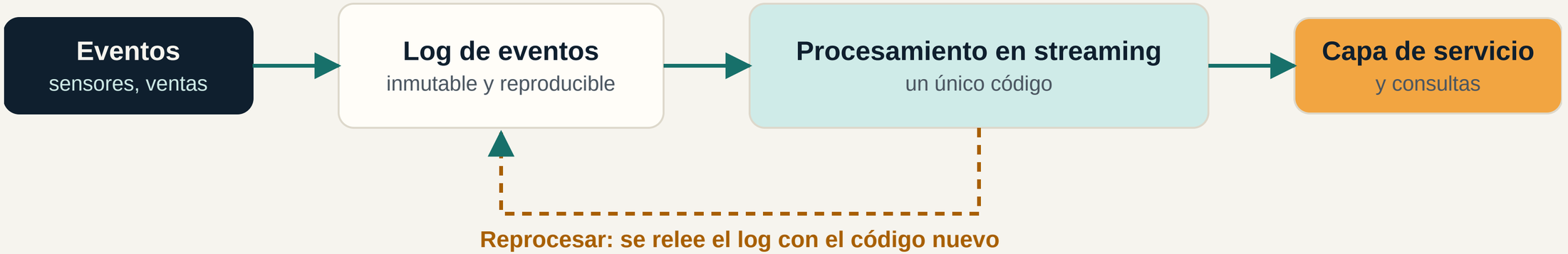
VENTANA DE TIEMPO

Se agrupan los eventos por intervalos (por ejemplo, cada minuto) para poder calcular acumulados en un flujo sin fin.

EVENTO TARDÍO

Un evento puede llegar después de cerrar su ventana: hay que decidir cuánto esperar antes de dar el resultado por bueno.

Arquitectura kappa: un único camino



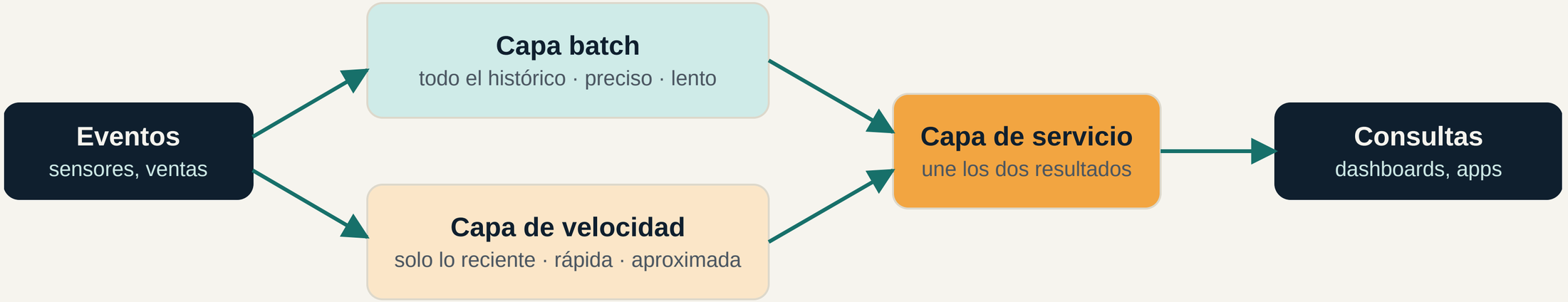
VENTAJAS

Una sola base de código y un solo motor. Más simple de operar y de razonar.

INCONVENIENTES

Reprocesar el histórico es releer todo el log. No encaja si el algoritmo por lotes es mucho mejor que el de streaming.

Arquitectura lambda: cuando un camino no basta



VENTAJAS

Resultados exactos con todo el histórico y respuesta rápida con lo último. Los fallos se corrigen recalculando el lote.

INCONVENIENTES

Hay que escribir y mantener la lógica dos veces (batch y streaming) y comprobar que ambas dan lo mismo.

Kappa o lambda: cuál elegir

	KAPPA	LAMBDA
Camino	Uno: streaming	Dos: lote y velocidad
Código	Una sola base	Dos bases que deben coincidir
Recalcular	Releer el log con código nuevo	Rehacer el lote completo
Encaja si	Lote y tiempo real dan lo mismo	Lote y tiempo real difieren
Ejemplo	Eventos de sensores	Informe final y marcador en directo

Empieza por la más simple: kappa. Pasa a lambda solo si lote y tiempo real necesitan cálculos distintos.

1.7 · PREGUNTA ABIERTA

¿Y si usamos varias a la vez?

APLICACIÓN

Tienda online

MySQL
Pedidos y pagos

MongoDB
Catálogo y opiniones

Redis
Sesiones y caché

Data lake
Fotos, logs y copias

¿Cómo se mantiene coherente el mismo dato cuando vive en varios sitios?

1.7 · PREGUNTA ABIERTA

Sincronizar los datos

¿Cuál es la fuente de verdad?

Si el precio cambia en MySQL, ¿quién actualiza la caché de Redis?

¿Cuándo debe coincidir?

Al instante o con unos segundos de retraso: consistencia inmediata o eventual.

¿Cómo viajan los cambios?

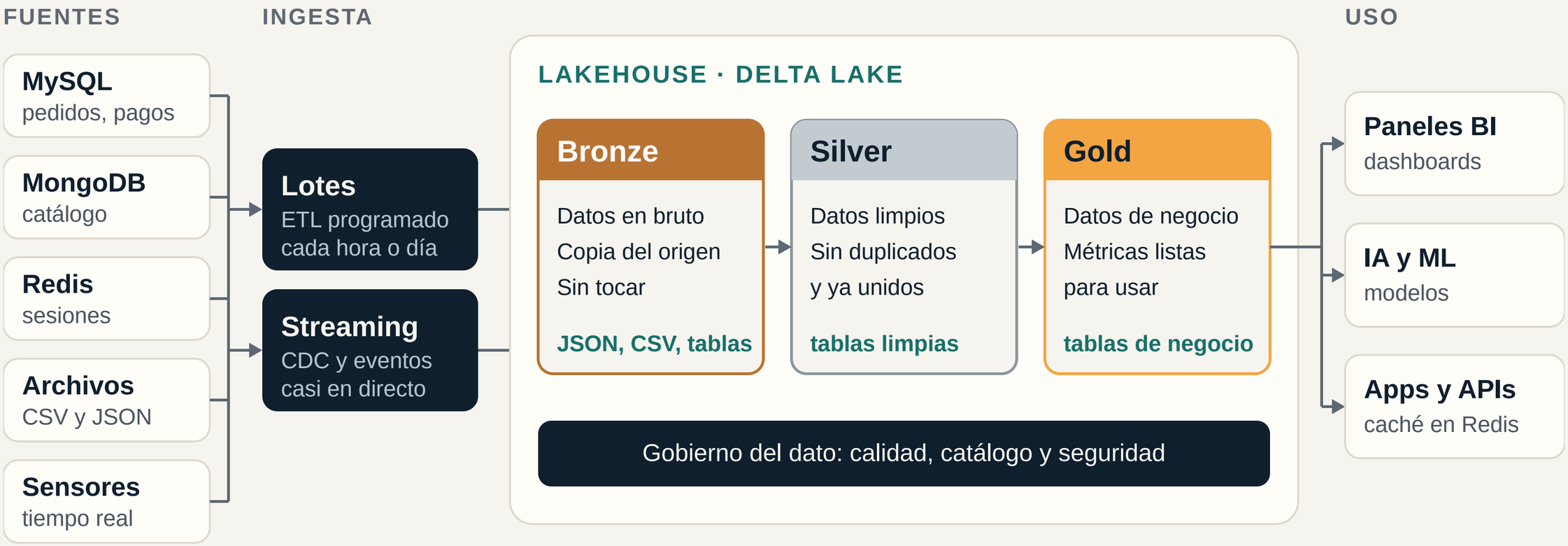
Colas de mensajes, procesos ETL o captura de cambios (CDC).

¿Qué pasa si algo falla?

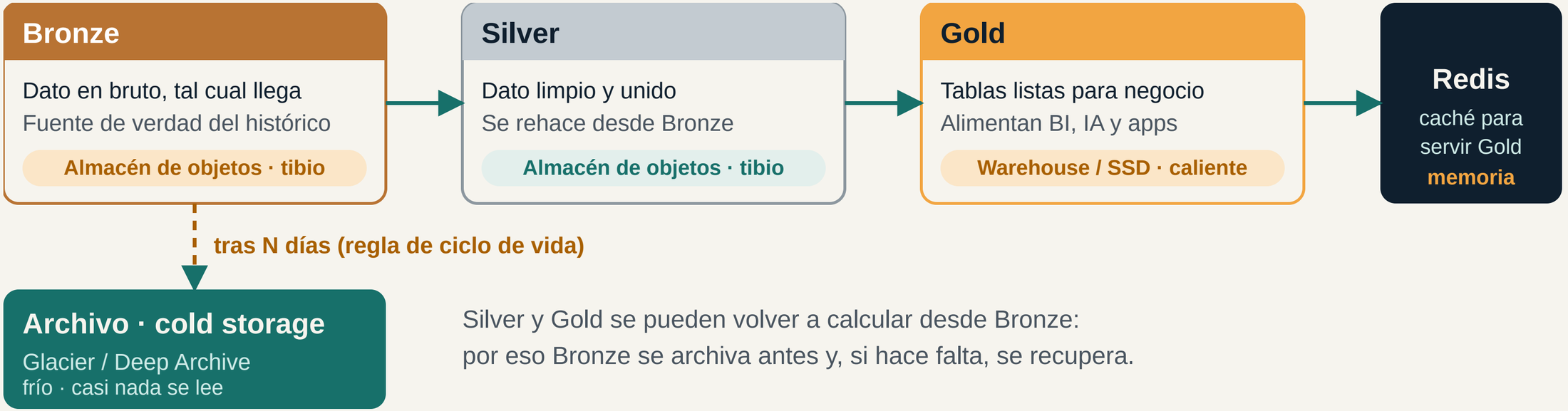
Reintentos, duplicados y mensajes que llegan desordenados.

Una respuesta habitual en la industria: la **arquitectura medallion**.

Arquitectura medallion



Medallion y capas de almacenamiento



Silver y Gold se pueden volver a calcular desde Bronze:
por eso Bronze se archiva antes y, si hace falta, se recupera.

1.7 · ARQUITECTURA

Medallion con el caso del Sporting

Bronze

Los JSON de StatsBomb tal cual: un fichero por partido.

Bloque 2: los leemos con pandas.

Silver

Eventos aplanados en una tabla de 128.813 filas, con tipos corregidos.

Bloque 2: sporting_eventos.csv.

Gold

Una fila por jugador y partido: goles, % de pases y xG.

Alimenta el dashboard y el modelo de ML.

Cada capa se puede reconstruir desde la anterior: si hay un error, se corrige y se vuelve a calcular.

Kappa, lambda y medallion no compiten

	KAPPA	LAMBDA	MEDALLION
Idea	Un camino: streaming	Dos caminos: lote y velocidad	Capas bronze, silver y gold
Responde a	¿Cómo proceso?	¿Cómo proceso?	¿Cómo organizo la calidad?
Coste	Reprocesar el log	Dos bases de código	Almacenar varias copias
Ejemplo	Eventos de sensores	Informe final y marcador en directo	Bronze: JSON; gold: informe

Kappa y lambda organizan el camino; medallion organiza la calidad. Se combinan.

Del dato a la decisión

1

Ingesta

Captura desde bases de datos, APIs, sensores o ficheros.

`Kafka · APIs · CSV`

2

Almacenamiento

Guardar a escala en data lakes y data warehouses.

`HDFS · S3 · Delta`

3

Procesamiento

Limpiar, transformar y analizar, en local o en clúster.

`pandas · Spark`

4

Visualización

Indicadores y cuadros de mando para decidir.

`Power BI · Plotly`

Recorreremos las cuatro fases con **un mismo dataset** a lo largo del curso.

La estrategia va primero, la arquitectura después

Una arquitectura excelente sin estrategia resuelve muy bien el problema equivocado.

Kappa, lambda y medallion son **medios**. La estrategia define el fin.

Las arquitecturas cambian con la tecnología; **las preguntas de negocio perduran**.

Primero se decide **para qué**; después, con qué construirlo.

La tecnología no es la estrategia

EL CÓMO · YA LO SABEMOS

- Bases de datos y data lakes
- Arquitectura y herramientas
- Seguridad y RGPD
- Calidad del dato

→

EL PARA QUÉ · LA ESTRATEGIA

- ¿Qué objetivo de negocio perseguimos?
- ¿Qué decisiones queremos mejorar?
- ¿Quién es responsable de cada dato?
- ¿Cómo medimos el valor?

SEÑALES DE QUE FALTA ESTRATEGIA

Proyectos que empiezan por la herramienta, no por la pregunta	Datos en silos: cada área con su versión	Pilotos que nunca llegan a producción	Nadie responde por la calidad del dato

Ocho piezas de una estrategia

1
Objetivos de negocio
Qué queremos conseguir con los datos.

2
Casos de uso
Qué decisiones mejorar, por orden de valor.

3
Datos y arquitectura
Dónde viven y cómo fluyen.

4
Gobierno y calidad
Quién es dueño de cada dato y qué calidad exige.

5
Seguridad y cumplimiento
Accesos, RGPD y ética.

6
Personas y cultura
Roles, formación y decisiones con datos.

7
Herramientas y presupuesto
Qué plataformas hacen falta y cuánto cuestan.

8
Medición del valor
KPIs y retorno de cada iniciativa.

Las tres primeras definen el **para qué**; el resto lo hace posible y lo sostiene.

De la pregunta a la hoja de ruta

1 Pregunta	2 Decisión	3 Datos	4 Equipo	5 Medida
¿Cómo fichar mejor con menos presupuesto?	Qué jugadores fichar y cuánto pagar.	Tablas, eventos de partido, informes y vídeo.	Analista, ingeniero de datos y ojeadores.	Rendimiento de cada fichaje frente a su coste.

HOJA DE RUTA · EJEMPLO

0–3 meses
Piloto con un solo caso.

→

3–12 meses
Plataforma y gobierno del dato.

→

12+ meses
Escalar a más áreas.

1.9 · SEGURIDAD DEL DATO

Un dato valioso es un dato atacado

Tres propiedades que hay que proteger, con tres preguntas prácticas debajo.

Confidencialidad

Solo ve el dato quien debe verlo.

Integridad

El dato no se cambia sin que se note.

Disponibilidad

El dato está cuando se necesita.

¿Quién puede entrar? ¿Cómo viaja el dato? ¿Cómo se guarda?

Cifrar: ilegible sin la clave

Ejemplo en Python con la librería cryptography (datos ficticios):

```python
from cryptography.fernet import Fernet
clave = Fernet.generate_key() # la clave es el secreto
f = Fernet(clave)
cifrado = f.encrypt("Ana López, 1980".encode())
print(cifrado) # b'gAAAA...' ilegible
print(f.decrypt(cifrado).decode()) # Ana López, 1980
```

En reposo

Disco, base de datos o bucket cifrados (AES-256 es lo habitual).

En tránsito

HTTPS/TLS: nadie lee el dato mientras viaja.

En uso

El dato descifrado en memoria: el punto más difícil.

Cifrar no es la única técnica

	QUÉ HACE	¿REVERSIBLE?	EJEMPLO
Cifrado	Transforma con una clave	Sí, con la clave	Disco de la base de datos
Hash	Huella de tamaño fijo	No	Guardar contraseñas (con sal)
Seudonimización	Cambia el identificador por un código	Sí, con la tabla aparte	IDs del Netflix Prize
Enmascarado	Oculta parte del dato	No	**** **** **** 1234
Tokenización	Cambia el dato por un token	Sí, solo desde la caja fuerte	Tarjeta de pago

Todas dependen de algo que hay que guardar bien: la clave, la sal o la caja fuerte.

De la seguridad perimetral al zero trust

MODELO CLÁSICO

El castillo y el foso

- Dentro de la red se confía
- Un cortafuegos protege el borde
- La VPN abre acceso a toda la red
- Si el atacante entra, se mueve libre

ZERO TRUST

Nunca confiar, siempre verificar

- Cada acceso se valida: identidad, dispositivo y contexto
- Acceso solo a la aplicación concreta, con mínimo privilegio
- Se asume que la brecha puede existir ya

Con zero trust, estar «dentro» de la red ya no da acceso a nada por sí solo.

Zero trust en la práctica: el caso de Zscaler

Usuario y dispositivo

Quién es, qué equipo usa y desde dónde

Punto de decisión

Verifica identidad, estado del equipo y política

Aplicación concreta

Solo esa; el resto de la red ni se ve

El enfoque de Zscaler

Su Zero Trust Exchange actúa de intermediario en la nube entre usuarios y aplicaciones: las aplicaciones no se publican en internet y son invisibles para quien no está autorizado.

En nuestra plataforma (ilustrativo)

El analista accede solo a gold; el ingeniero de datos, a bronze y silver; los datos personales, solo con una razón y registrados.

Las cuatro fases de un ataque digital

Modelo de cuatro fases de Zscaler, alineado con MITRE ATT&CK.

1

Encontrar

Atacante: Busca aplicaciones y servidores expuestos a internet.

Defensa: Reducir la superficie: no exponer lo que no hace falta.

2

Comprometer

Atacante: Entra con phishing, malware o credenciales robadas.

Defensa: Filtrar correo y web, inspeccionar el tráfico, MFA.

3

Moverse

Atacante: Salta de un sistema a otro buscando datos y privilegios.

Defensa: Segmentar: cada usuario ve solo su aplicación.

4

Robar

Atacante: Reúne y extrae datos, a veces cifrándolos para pedir rescate.

Defensa: Prevención de pérdida de datos (DLP), cifrado y alertas.

1.9 · SEGURIDAD DEL DATO

Un ataque, paso a paso, y dónde se corta

	QUÉ PASA (ESCENARIO INVENTADO)	QUÉ LO CORTA
1 · Encontrar	Un panel del data lake queda publicado sin autenticación	No exponerlo; acceso solo por identidad
2 · Comprometer	Un correo falso roba la contraseña de un analista	Doble factor (MFA) y filtro de correo
3 · Moverse	Con esa cuenta llega a bronze y silver, con datos personales	Mínimo privilegio: el analista solo ve gold
4 · Robar	Descarga millones de filas sin que salte ninguna alerta	Cifrado, límites de descarga y alertas

Al atacante le hace falta que todo salga bien; a la defensa, que un solo control funcione.

Seis controles para llevarse

Una lista corta para aplicar mañana mismo.

Cifrar

Reposo, tránsito y gestión de claves.

Seudonimizar y enmascarar

Que cada persona vea solo lo que necesita.

Mínimo privilegio

Acceso a la capa y al dato justos.

Zero trust

Verificar cada acceso, no la red.

Registrar y vigilar

Logs y alertas por descargas raras.

Plan de respuesta

Qué hacer, y a quién avisar en 72 h.

Pregunta para el grupo: **¿dónde está vuestro dato más sensible y quién puede llegar a él?**

Protección de datos

RGPD

Reglamento europeo 2016/679: consentimiento, finalidad y minimización de datos.

LOPDGDD

Ley Orgánica 3/2018: adapta el RGPD en España y recoge los derechos digitales.

Privacidad desde el diseño

Minimizar, seudonimizar y proteger los datos desde el inicio del proyecto.

Derechos de las personas: **acceso, rectificación, supresión y oposición**.

El Netflix Prize: el escenario

1 M$
para quien mejorase un 10 % el sistema de recomendación

100 M
de valoraciones publicadas

480.000
usuarios, sin nombre

17.770
películas

Netflix publica los datos «anonimizados» → Equipos de todo el mundo entrenan modelos → Gana quien mejor predice las notas

El reto: predecir qué nota pondría un usuario a una película que aún no ha visto.

Así eran los datos

USUARIO	PELÍCULA	NOTA	FECHA
4821	1203	5	2005-03-14
4821	877	4	2005-03-14
4821	3090	5	2005-05-02
4821	412	5	2005-07-19
9370	1203	3	2005-04-01
9370	2210	4	2005-04-01
5512	877	1	2004-11-23
5512	3090	2	2004-12-09

NO HAY

Nombre

Email

Dirección

Teléfono

SÍ HAY

Un número por usuario

Qué películas valoró

Qué nota puso

Qué día

DEBATE · 5 MINUTOS

Antes de seguir, discutid en parejas

1. ¿Hay aquí algún dato personal?
2. ¿Cómo podría alguien averiguar quién es el usuario 4821?
3. ¿Qué es lo peor que podría pasarle a esa persona?

Pista: piensa en qué otras listas de películas con nota y fecha existen en internet.

El cruce que rompe el anonimato

NETFLIX · USUARIO 4821 (ANÓNIMO)

El faro azul	5	2005-03-14
Noche en Lisboa	4	2005-03-14
Orgullo en la plaza	5	2005-05-02
Huelga general	5	2005-07-19

⇄

Misma nota, misma fecha

IMDB · PERFIL PÚBLICO DE MARTA G.

El faro azul	5	2005-03-15
Noche en Lisboa	4	2005-03-14

Solo valoró en público estas dos. Las otras dos las vio en privado.

Dos coincidencias bastan: el usuario 4821 es Marta G. Y ahora sabemos qué más vio.

1.10 · CASO REAL · NETFLIX

El Netflix Prize y la anonimización

1. En 2006 Netflix lanza un concurso de 1 M$ y publica unos 100 millones de valoraciones de 480.000 usuarios, con identificadores «anónimos».
2. Los investigadores Narayanan y Shmatikov cruzan esas valoraciones con las públicas de IMDb y reconocen a personas concretas.
3. Con muy poca información externa bastaba para encontrar un registro, y se deducían preferencias políticas y otros datos sensibles.

CONSECUENCIAS

Cancelado

el segundo concurso, en marzo de 2010

Diciembre de 2009: demanda colectiva de cuatro usuarios, que Netflix resuelve en marzo de 2010.

La FTC preguntó por la privacidad de los miembros y Netflix llegó a un acuerdo.

LECCIÓN Quitar el nombre no es anonimizar: al cruzar fuentes, los datos vuelven a señalar a una persona.

1.10 · CASO REAL · STRAVA

El mapa que delató bases militares

1. Strava publica un mapa de calor global con la actividad de sus usuarios (noviembre de 2017).
2. Soldados y personal de seguridad usaban la app en bases y patrullas: sus rutas dibujaban instalaciones y rutinas.
3. En enero de 2018 un estudiante, Nathan Ruser, lo hace público. El usuario tenía que desactivar la opción para quedar fuera.

CONSECUENCIAS

+1.000 M

de rutas registradas en el mapa

Agosto de 2018: el Pentágono prohíbe la geolocalización al personal desplegado.

Strava recuerda que las actividades privadas y las zonas de privacidad quedaban excluidas.

LECCIÓN Datos individuales inofensivos, sumados, revelan un patrón sensible. La configuración por defecto importa.

Fuentes: Help Net Security, NPR · Big Data e IA · Bloque 1

1.10 · CASO REAL · FACEBOOK

Cambridge Analytica y los datos de los amigos

1. Una app de terceros recoge datos personales de millones de usuarios de Facebook.
2. También accede a datos de sus amigos, aunque ellos tuvieran ajustes de privacidad más restrictivos.
3. Los datos acaban en la consultora Cambridge Analytica. El escándalo estalla en 2018.

CONSECUENCIAS

5.000 M$

sanción de la FTC a Facebook (julio de 2019)

La mayor impuesta por la FTC por violar la privacidad.

Orden de 20 años: comité de privacidad y revisión de cada producto nuevo.

LECCIÓN El consentimiento de una persona no cubre los datos de otras, y cada dato tiene una finalidad.

1.10 · CASO REAL · EQUIFAX

La brecha de una agencia de crédito

1. En 2017 atacantes acceden a los sistemas de Equifax, una de las grandes agencias de crédito de EE. UU.
2. Quedan expuestos los datos de unos 147 millones de personas, incluidos números de la Seguridad Social y carnés de conducir.
3. Los reguladores concluyen que no protegió bien los datos ni fue claro sobre su seguridad.

CONSECUENCIAS

700 M$

acuerdo de 2019 con FTC, CFPB y estados (hasta esa cifra)

300 M$ de fondo para afectados, que podía subir a 425 M$.

175 M$ a los estados y 100 M$ a la CFPB.

LECCIÓN Guardar datos sensibles es una responsabilidad: conserva solo lo necesario y protégelo.

1.10 · CASO REAL · ESPAÑA

Mercadona y el reconocimiento facial

1. Mercadona prueba cámaras de reconocimiento facial en sus tiendas para detectar a personas con medidas judiciales en su contra.

2. El sistema captaba datos biométricos de clientes y trabajadores y los conservó casi un año, hasta mayo de 2021.

3. La AEPD concluye que no había base jurídica, que confundía utilidad con necesidad y que era desproporcionado.

CONSECUENCIAS

2,5 M€

multa de la AEPD (julio de 2021)

Mercadona retira el sistema y cierra la prueba piloto.

Resolución PS-00120-2021 de la AEPD.

LECCIÓN Que algo sea útil no lo hace necesario ni proporcionado. Los datos biométricos son categoría especial.

Fuentes: Genbeta, OECD.AI · Big Data e IA · Bloque 1

Cinco casos, cinco principios

CASO	AÑOS	QUÉ FALLÓ	PRINCIPIO
Netflix Prize	2008–10	Datos «anonimizados» re-identificados cruzándolos con IMDb	**Anonimización real**
Strava	2017–18	Sumar rutas inocuas reveló bases militares	**Privacidad por defecto**
Equifax	2017–19	Datos sensibles de unos 147 M de personas mal protegidos	**Seguridad y minimización**
Facebook · Cambridge Analytica	2018–19	Datos de amigos compartidos sin su consentimiento	**Consentimiento y finalidad**
Mercadona	2021	Biometría masiva sin base jurídica	**Proporcionalidad**

1.10 · ZONA GRIS

Usos dudosos de los datos

SEGUROS

¿Debe cambiar tu prima según tus pasos o tu forma de conducir?

PRECIOS

¿Puede una tienda cobrarte distinto según tu móvil o tu barrio?

CRÉDITO

¿Es justo denegar un préstamo por tu código postal?

MENORES

¿Hasta dónde se puede perfilar a menores o personas vulnerables?

UBICACIÓN

¿Vender datos de ubicación «anonimizados» a terceros?

EMPLEO

¿Revisar las redes sociales de un candidato antes de contratarlo?

Que sea legal no significa que sea ético.

Seis preguntas antes de usar un dato

1. ¿Para qué lo recojo?
2. ¿Lo sabe y lo aceptó quien lo dio?
3. ¿Necesito todo esto, o menos sirve?
4. ¿Se puede re-identificar o cruzar con otros datos?
5. ¿Quién puede salir perjudicado?
6. ¿Qué pasa si se filtra?

PARA DEBATIR Elige un dilema de la slide anterior y aplícale las seis preguntas.

Seis preguntas sobre cualquier dato

Completitud
¿Faltan valores?

Exactitud
¿Son correctos?

Consistencia
¿Coinciden entre fuentes?

Unicidad
¿Hay duplicados?

Actualidad
¿Están al día?

Validez
¿Tienen el formato esperado?

Un informe o un modelo solo es tan bueno como los datos que lo alimentan.

Un fallo por dimensión

DIMENSIÓN	CASO PRÁCTICO	CONSECUENCIA
Completitud	Clientes sin email	No reciben avisos de fraude
Exactitud	Edad de 212 años	Falsea medias y segmentos
Consistencia	«Gijón» y «gijon» en dos sistemas	Un cliente cuenta como dos
Unicidad	Cliente dado de alta dos veces	Doble envío y totales inflados
Actualidad	Dirección de hace ocho años	Cartas devueltas, decisión errónea
Validez	Email «luis@mail» sin dominio	El correo rebota y la carga falla

EJERCICIO · 3 MINUTOS

¿Cuántos fallos de calidad ves?

ID	NOMBRE	EMAIL	EDAD	CIUDAD	ACTUALIZADO
1	Ana Ruiz	ana@mail.com	34	Gijón	2025-10
2	Luis Pérez	luis@mail	29	Oviedo	2025-09
3	Ana Ruiz	ana@mail.com	34	Gijón	2025-10
4	Marta Gil	marta@mail.com	212	Gijón	2025-08
5	Pedro Sosa		41	Avilés	2025-10
6	Carla Ríos	carla@mail.com	38	gijon	2025-07
7	Jorge Vega	jorge@mail.com	45	Oviedo	2016-05

SOLUCIÓN

Seis fallos, seis dimensiones

ID	NOMBRE	EMAIL	EDAD	CIUDAD	ACTUALIZADO	FALLO
1	Ana Ruiz	ana@mail.com	34	Gijón	2025-10	Sin fallos
2	Luis Pérez	luis@mail	29	Oviedo	2025-09	**Validez**
3	Ana Ruiz	ana@mail.com	34	Gijón	2025-10	**Unicidad**
4	Marta Gil	marta@mail.com	212	Gijón	2025-08	**Exactitud**
5	Pedro Sosa		41	Avilés	2025-10	**Completitud**
6	Carla Ríos	carla@mail.com	38	gijon	2025-07	**Consistencia**
7	Jorge Vega	jorge@mail.com	45	Oviedo	2016-05	**Actualidad**

ACTIVIDAD · 20 MINUTOS

Debate: ¿dónde hay Big Data en tu día a día?

1. Elige una empresa o servicio que uses (banca, streaming, transporte, tiendas).
2. ¿Qué V predomina en su caso?
3. ¿Qué riesgo ético o de privacidad ves?

ACTIVIDAD · EJEMPLO

Así se ve un caso completo: un banco

Servicio	Un banco: tarjeta, banca online y app móvil
Datos que genera	Pagos, ubicación del comercio, movimientos en la app, historial de crédito
V predominante	Velocidad: cada pago se evalúa en milisegundos para detectar fraude
Riesgo ético o de privacidad	Un scoring que deniega créditos con sesgo, o el rastreo de hábitos de gasto
Calidad que vigilaría	Actualidad y exactitud: un ingreso desfasado cambia la decisión

TU TURNO · 10 MINUTOS

Rellena la plantilla con tu caso

Servicio

Datos que genera

V predominante

Riesgo ético o de privacidad

Calidad que vigilaría

02

Programación para el tratamiento de datos

Bloque 2 · 3 horas

Python para datos

pandas

Tablas (DataFrame) para cargar, filtrar, agrupar y limpiar datos.

```
import pandas as pd
```

NumPy

Arrays numéricos y cálculo rápido; es la base de pandas.

```
import numpy as np
```

Gráficos

Matplotlib, Seaborn y Plotly para explorar y comunicar.

```
import seaborn as sns
```

Google Colab

Cuadernos de Python en el navegador, gratis y sin instalar nada.

- Celdas de código y de texto
- Librerías habituales ya instaladas
- Se guarda en Drive y se comparte con un enlace

ABRIR EL NOTEBOOK

1. Entra en colab.research.google.com
2. Archivo, Subir cuaderno
3. Elige el fichero 01 y ejecuta celda a celda

Importar librerías y fijar parámetros

python · Colab

```python
import json, requests
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

BASE = "https://raw.githubusercontent.com/statsbomb/open-data/master/data"
EQUIPO = "Sporting Gijón"
```

import … as …

pd, np y plt son alias que usa todo el mundo: así el código de otros se lee igual.

requests

Descarga ficheros por HTTP: así traemos el JSON público de StatsBomb.

Constantes

BASE y EQUIPO en mayúsculas: los parámetros viven en un solo sitio.

Ciclo de tratamiento de datos

1

Cargar

Leer el fichero.

`read_csv()`

2

Explorar

Entender qué hay.

`describe()`

3

Limpiar

Corregir problemas.

`dropna()`

4

Transformar

Crear variables.

`map()`

5

Guardar

Reutilizar el resultado.

`to_csv()`

El dataset limpio del paso 5 se reutiliza en los siguientes bloques del curso.

Nulos, tipos y duplicados

PROBLEMA	DETECTAR	CORREGIR
Valores nulos	`df.isnull().sum()`	`df['arbitro'].fillna('Desconocido')`
Tipos erróneos	`df.info()`	`pd.to_datetime(df['fecha'])`
Duplicados	`df.duplicated().sum()`	`df.drop_duplicates()`

Real Sporting de Gijón 2015/16

Eventos de los 38 partidos del Sporting en La Liga: pases, tiros, duelos y más.

- Unos 129.000 eventos en 38 partidos
- JSON anidado que convertimos en tablas
- Cada evento con su posición en el campo

PREGUNTAS GUÍA

1. ¿Qué nulos son errores y cuáles son normales?
2. ¿Qué significa un pase sin resultado?
3. ¿Cuántos puntos sumó el Sporting?

2.5 · CÓDIGO: INGESTA DE PARTIDOS

De JSON anidado a tabla

python · Colab

```python
def leer_json(ruta):
    r = requests.get(f"{BASE}/{ruta}", timeout=60)
    r.raise_for_status() # falla si la descarga no fue bien
    return r.json()

partidos_raw = pd.json_normalize(leer_json("matches/11/27.json"))
partidos_raw.shape
```

Una función

Se escribe una vez y se reutiliza 40 veces: catálogo, partidos y eventos.

json_normalize

Aplana objetos anidados: home_team pasa a la columna home_team.home_team_name.

11 y 27

Identificadores de La Liga y de la temporada 2015/16 en el catálogo de StatsBomb.

Tipos, nulos y vista del Sporting

python · Colab

```
es_local = sporting["home_team.home_team_name"] == EQUIPO
partidos = pd.DataFrame({
    "fecha": pd.to_datetime(sporting["match_date"]),
    "goles_favor": np.where(es_local, sporting["home_score"], sporting["away_score"]),
    "goles_contra": np.where(es_local, sporting["away_score"], sporting["home_score"]),
    "arbitro": sporting["referee.name"].fillna("Desconocido"),
})
```

to_datetime

Convierte texto en fecha real: permite ordenar y restar fechas.

np.where

Condición ? valor si True : valor si False. Aquí elige goles según seas local.

fillna

Rellena un nulo con un valor con sentido; «Desconocido» no inventa un árbitro.

Crear variables nuevas: resultado y puntos

python · Colab

```
partidos["resultado"] = np.select(
    [partidos["goles_favor"] > partidos["goles_contra"],
     partidos["goles_favor"] == partidos["goles_contra"]],
    ["V", "E"], default="D")
partidos["puntos"] = partidos["resultado"].map({"V": 3, "E": 1, "D": 0})
partidos["puntos"].sum() # 39
```

np.select

Lista de condiciones y de valores: gana la primera que se cumple; default para el resto.

map

Cambia cada valor por otro usando un diccionario: V→3, E→1, D→0.

39 puntos

Resultado: 10 victorias, 9 empates y 19 derrotas.

Una función que aplana cada evento

python · Colab

```python
def aplanar_evento(e, match_id):
    fila = {"match_id": match_id, "type": e["type"]["name"],
            "player": e.get("player", {}).get("name")}
    if "location" in e:
        fila["x"], fila["y"] = e["location"]
    return fila
```

Variedad

Cada tipo de evento trae campos distintos: un pase tiene destinatario, un tiro tiene xG.

e.get(…, {})

get devuelve un valor por defecto si el campo no existe, en lugar de dar error.

Por qué una función

Controlamos qué sacamos de cada estructura anidada, en vez de aplanarlo todo.

El volumen: 38 ficheros, una tabla

python · Colab

```python
filas = []
for mid in partidos["match_id"]:
    for e in leer_json(f"events/{mid}.json"):
        filas.append(aplanar_evento(e, mid))

eventos = pd.DataFrame(filas)
eventos.shape # (128813, columnas)
```

Bucle doble

Para cada partido, para cada evento: se añade una fila a la lista.

Lista → tabla

Construir la lista primero y crear el DataFrame al final es mucho más rápido.

128.813 eventos

Es el volumen. Tarda uno o dos minutos: es la descarga, no pandas.

2.5 · CÓDIGO: CALIDAD DE LOS EVENTOS

Nulos que significan algo

python · Colab

```
(eventos.isnull().mean() * 100).round(1).sort_values(ascending=False).head(15)

pases = eventos[eventos["type"] == "Pass"]
pases["pass_outcome"].isnull().sum()
pases["pass_outcome"].value_counts(dropna=False)
```

Nulos estructurales

pass_* solo existe en los pases y shot_* en los tiros: no faltan, no aplican.

pass_outcome

Vacío = pase completo. Solo se rellena cuando algo sale mal.

dropna=False

value_counts oculta los nulos salvo que se lo pidas.

2.5 · CÓDIGO: LIMPIAR EVENTOS

Dar sentido a los nulos y crear columnas auxiliares

python · Colab

```
es_pase = eventos["type"] == "Pass"
eventos.loc[es_pase & eventos["pass_outcome"].isnull(),
            "pass_outcome"] = "Completo"

eventos["es_sporting"] = eventos["team"] == EQUIPO
eventos["pase_completo"] = np.where(
    es_pase, eventos["pass_outcome"] == "Completo", np.nan)
```

loc[filas, columna]

Selecciona y asigna a la vez; & combina dos condiciones.

Columna booleana

es_sporting separa los dos equipos para comparar sin repetir el filtro.

pase_completo

1 si completo, 0 si no, vacío si no es pase: su media es el % de acierto.

Describir y comparar con groupby

python · Colab

```
tiros = eventos[eventos["type"] == "Shot"]
tiros[["shot_xg", "x", "y"]].describe()

pases = eventos[eventos["type"] == "Pass"]
resumen = pases.groupby("team")["pase_completo"].agg(
    pases="size", acierto="mean")
resumen["acierto"] = (resumen["acierto"] * 100).round(1)
```

describe()

Cuenta, media, desviación, mínimo, cuartiles y máximo de cada columna numérica.

groupby

Dividir por grupo, aplicar una función y combinar el resultado en una tabla.

Comparar con cuidado

El Sporting sale en 38 partidos y cada rival en 2: se compara el %, no el número.

Primeros gráficos

Histograma

Una variable numérica

```
pases['pass_length'].hist()
```

Barras

Una variable categórica

```
eventos['type'].value_counts().head(8).plot(kind='bar')
```

Boxplot

Numérica según grupo

```
tiros.boxplot('shot_xg', by='shot_outcome')
```

Tres gráficos en una sola figura

python · Colab

```
fig, axs = plt.subplots(1, 3, figsize=(18, 4.5))

pases["pass_length"].hist(bins=40, ax=axs[0])
eventos["type"].value_counts().head(8).plot(kind="bar", ax=axs[1])
tiros.boxplot("shot_xg", by="shot_outcome", ax=axs[2], rot=45)

plt.tight_layout()
plt.show()
```

subplots

Una figura con 1 fila y 3 columnas; ax=axs[i] elige en qué hueco se dibuja.

Histograma y barras

Histograma: cómo se reparte una variable numérica. Barras: cuántos hay de cada categoría.

Boxplot

La caja contiene el 50 % central; la línea es la mediana; los puntos son atípicos.

Del gráfico al dashboard

Informe

Cuenta una historia a quien lo lee. Se explica y se archiva.

Dashboard

Monitoriza indicadores de un vistazo. Se mira cada día o semana.

Exploración

El analista la hace para entender los datos. Es el notebook.

ANTES DE DISEÑAR

¿Quién lo va a mirar?

¿Qué decisión tomará con él?

¿Cada cuánto lo consultará?

Cinco reglas para enseñar datos

1	**Una idea por gráfico**	El título dice la conclusión, no el nombre de la variable.
2	**El gráfico para la pregunta**	Comparar, evolucionar, distribuir o relacionar piden gráficos distintos.
3	**Eje honesto**	Las barras empiezan en cero. Truncar el eje exagera.
4	**Menos tinta**	Quita rejillas, bordes y colores que no cuentan nada.
5	**Contexto**	Compara con un objetivo o con el periodo anterior.

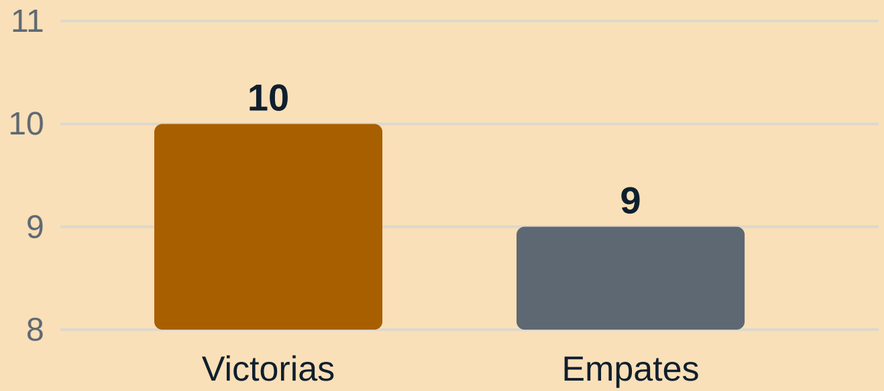
¿Qué gráfico para qué pregunta?

PREGUNTA	GRÁFICO	EJEMPLO CON EL SPORTING
Comparar categorías	**Barras**	Tiros del Sporting y sus rivales
Ver una evolución	**Líneas**	Puntos acumulados por jornada
Ver cómo se reparten los valores	**Histograma**	Longitud de los pases
Relacionar dos variables	**Dispersión**	xG de un tiro y si acaba en gol
Partes de un total	**Barras apiladas**	Victorias, empates y derrotas

El mismo dato, dos historias

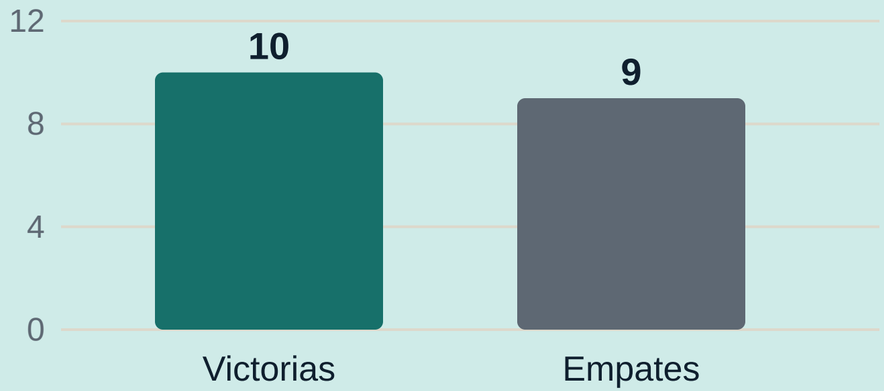
Eje desde 8

Parece que hay el doble de victorias.



Eje desde 0

Casi las mismas: 10 y 9.



Un dashboard con nuestros datos

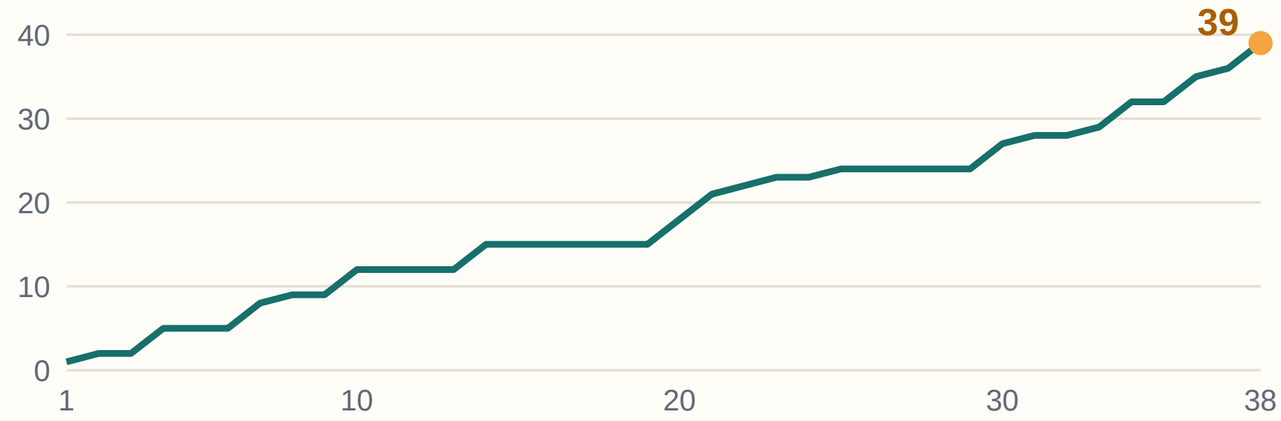
PUNTOS
39
10 V · 9 E · 19 D

GOLES
40 – 62
a favor – en contra

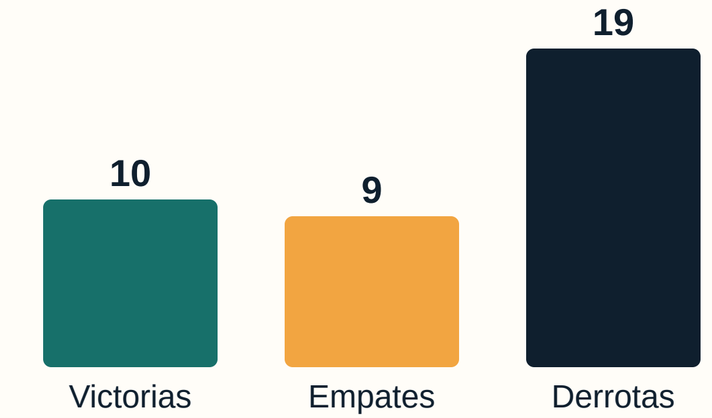
TIROS
380 – 546
Sporting – rivales

xG
41,0 – 58,3
creado – concedido

Puntos acumulados por jornada



Resultados en 38 partidos



Puntos acumulados y local frente a visitante

python · Colab

```
partidos["puntos_acum"] = partidos["puntos"].cumsum()

plt.plot(partidos["jornada"], partidos["puntos_acum"], marker="o")
plt.title("Puntos acumulados del Sporting (La Liga 2015/16)")
plt.xlabel("Jornada"); plt.ylabel("Puntos")

partidos.groupby("local")[["puntos", "goles_favor", "goles_contra"]].sum()
```

cumsum

Suma acumulada: pasa de los puntos de cada partido a la evolución de la temporada.

Gráfico de líneas

Es el gráfico para una evolución en el tiempo (regla 2 de los dashboards).

Título

Mejor si dice la conclusión que el nombre de la variable (regla 1).

Herramientas: de menos a más código

SIN CÓDIGO

Looker Studio

Informes conectados a hojas de cálculo y bases de datos.

SIN CÓDIGO

Power BI

Muy extendido en empresa. Power BI Desktop tiene versión gratuita.

SIN CÓDIGO

Tableau

Visualización avanzada. Tableau Public es gratuito.

CON PYTHON

Plotly y Streamlit

Dashboards web a partir de un notebook. Código abierto.

EN LA PLATAFORMA

Databricks

Dashboards sobre las tablas del lakehouse.

TIEMPO REAL

Grafana

Monitorización de métricas en directo. Código abierto.

PARA ELEGIR ¿Quién lo usa? ¿Datos en vivo o diarios? ¿Cuánto cuesta mantenerlo?

EJERCICIO · 10 MINUTOS

Diseña tu dashboard

El director deportivo del Sporting quiere seguir al equipo cada semana. Dibuja su dashboard en papel.

Audiencia	¿Quién lo mira?
Decisión	¿Qué decide con él?
Tres indicadores	¿Cuáles y por qué?
Gráficos	¿Cuál para cada uno?
Lo que dejas fuera	¿Qué no incluirías?

Guardar los datasets limpios en CSV

python · Colab

```python
import os

partidos.to_csv("sporting_partidos.csv", index=False)
eventos.to_csv("sporting_eventos.csv", index=False)

for f in ["sporting_partidos.csv", "sporting_eventos.csv"]:
    print(f, os.path.getsize(f) / 1024, "KB")
```

index=False

No guarda el número de fila como columna extra.

CSV

Formato universal: lo leerán Spark, las herramientas de BI y los modelos.

Colab es temporal

Los ficheros se pierden al cerrar la sesión: descárgalos o guárdalos en Drive.

EJERCICIO · 30 MINUTOS

Ahora te toca a ti

1. Calcula el porcentaje de pases completos del Sporting y compáralo con el de sus rivales.
2. ¿Qué jugador del Sporting hizo más tiros y cuál más goles?
3. Elige dos variables que creas que influyen en ganar un partido y justifícalo con un gráfico.
4. ¿Por qué sería un error aplicar dropna() sobre pass_outcome?

EJERCICIO 1 · OPCIONES

Pases completos: tres formas de calcularlo

La columna pase_completo vale 1 o 0, así que su media es el porcentaje de pases completos.

A Agrupar por equipo

La más corta: una línea separa Sporting y rivales.

```
pases.groupby("es_sporting")["pase_
completo"].mean()
```

B Dos tablas

Más pasos pero más fácil de seguir: filtras cada equipo y calculas su media.

```
pases[pases["es_sporting"] == True]
["pase_completo"].mean()
```

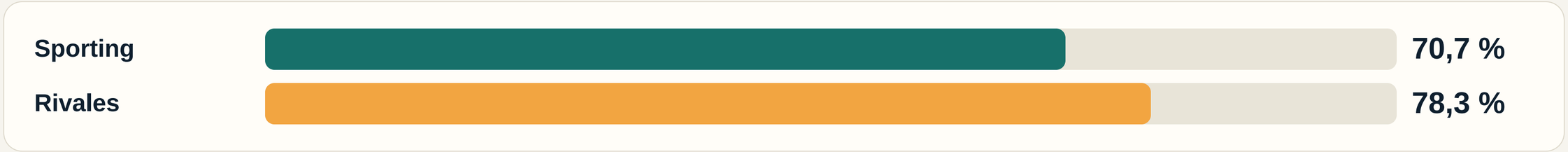
C Gráfico de barras

La misma cuenta, pero la diferencia se ve a simple vista.

```
porcentajes.plot(kind="bar")
```

EJERCICIO 1 · CONCLUSIÓN

El Sporting pasó peor que sus rivales



QUÉ QUEREMOS QUE SE LLEVEN

- Unos 8 puntos menos de acierto: sugiere un juego más directo y con más riesgo (balones largos) que el de posesión con pase corto.
- Cuidado con la lectura: los rivales incluyen equipos de mucha posesión y de cada uno solo hay 2 partidos. Es una pista, no una prueba.
- Comparamos porcentajes y no número de pases, porque el Sporting aparece en 38 partidos y cada rival en 2.

EJERCICIO 1 · CÓDIGO DE LA SOLUCIÓN

% de pases completos: Sporting y rivales

python · Colab

```
resultado = pases.groupby("es_sporting")["pase_completo"].mean() * 100
resultado.round(1)

resultado.plot(kind="bar")
plt.ylabel("% de pases completos")
plt.ylim(0, 100)
```

mean() de 1/0

La media de una columna de unos y ceros es una proporción.

ylim(0, 100)

Eje que empieza en cero: regla 3, el eje honesto.

Resultado

Sporting 70,7 % frente a rivales 78,3 %.

EJERCICIO 2 · OPCIONES

Tiros y goles por jugador: cuatro opciones

Cada fila de tiros_sp es un tiro, así que contar filas por jugador es contar tiros. La tabla goles solo tiene los que acabaron en gol.

A groupby + size

Agrupa por jugador y cuenta filas; después ordena de mayor a menor.

```
tiros_sp.groupby("player").size()
  .sort_values(ascending=False)
```

B value_counts

Hace lo mismo en una sola línea y ya ordena.

```
tiros_sp["player"].value_counts().head(5)
```

C Tabla con agg

Tiros, goles y xG de cada jugador en una sola tabla.

```
tiros_sp.groupby("player").agg(
  tiros=("event_id","size"), ...)
```

D Gráfico de barras

Los 8 jugadores que más tiran, en horizontal.

```
...value_counts().head(8)
  .plot(kind="barh")
```

EJERCICIO 2 · CONCLUSIÓN

Tirar mucho no es marcar mucho

JUGADOR	TIROS	GOLES	XG
Halilović	59	3	4,2
J. Rodríguez	56	5	3,2
Sanabria	52	11	11,4
C. Castro	25	7	5,4

QUÉ QUEREMOS QUE SE LLEVEN

- Más tiros: Halilović (59). Más goles: Sanabria (11).
- Halilović tira más que nadie y marca 3; Sanabria tira menos y marca 11.
- El xG separa cantidad de calidad: los tiros de Sanabria eran ocasiones claras (11,4 xG) y las acabó.

EJERCICIO 2 · CÓDIGO DE LA SOLUCIÓN

Tiros, goles y xG por jugador

python · Colab

```python
tiros_sp = tiros[tiros["team"] == EQUIPO]
tabla = tiros_sp.groupby("player").agg(
    tiros=("event_id", "size"),
    goles=("shot_outcome", lambda s: (s == "Goal").sum()),
    xg=("shot_xg", "sum"),
).sort_values("tiros", ascending=False)
tabla.head(10).round(2)
```

agg con nombre

nombre=(columna, función): varias métricas en una sola pasada.

lambda

Función anónima de una línea: cuenta cuántos tiros tienen resultado Goal.

xG acumulado

Separa tirar mucho de tirar bien: Sanabria 11,4 xG y 11 goles.

EJERCICIO 3 · PREPARAR LOS DATOS

Una tabla con una fila por partido

python · Colab

```
pases_sp = pases[pases["es_sporting"]]
por_partido = partidos.set_index("match_id")[["resultado", "puntos"]].copy()
por_partido["tiros"] = tiros_sp.groupby("match_id").size()
por_partido["xg"] = tiros_sp.groupby("match_id")["shot_xg"].sum()
por_partido["pases"] = pases_sp.groupby("match_id").size()
por_partido["acierto_pase"] = pases_sp.groupby("match_id")["pase_completo"].mean() * 100
por_partido = por_partido.fillna(0)
```

Unidad de análisis

Para preguntar qué influye en ganar, cada fila debe ser un partido: 38 filas.

Alineación por índice

Al asignar una serie con match_id, pandas la encaja con el índice de la tabla.

fillna(0)

Un partido sin tiros no aparece en el groupby: se rellena con 0, que aquí sí es correcto.

EJERCICIO 3 · OPCIONES

¿Qué influye en ganar? Cuatro formas de mirarlo

Primero se crea una tabla con una fila por partido. Luego se elige una variable (xg, tiros, pases o acierto_pase) y una opción.

A Media por resultado

Valor medio de la variable en victorias, empates y derrotas.

```
por_partido.groupby("resultado")
  [variable].mean()
```

B Boxplot

La caja enseña dónde está la mayoría de los partidos de cada resultado.

```
por_partido.boxplot(variable,
  by="resultado")
```

C Dispersión

Cada punto es un partido: variable frente a puntos.

```
plt.scatter(por_partido[variable],
  por_partido["puntos"])
```

D Correlación

Un número por variable: cerca de 0 no influye; cerca de 1 influye mucho.

```
por_partido[[...]].corr()
  ["puntos"]
```

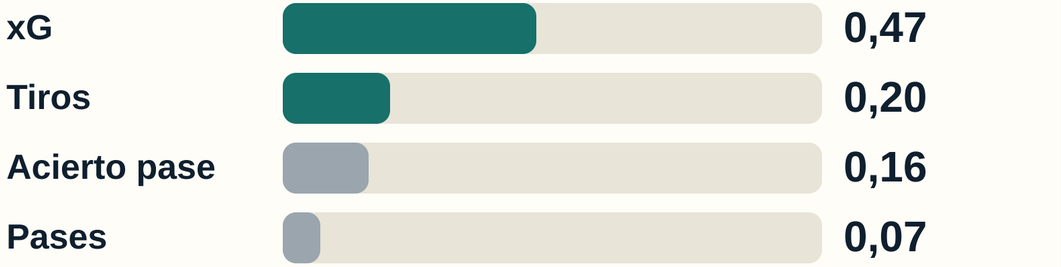
EJERCICIO 3 · CONCLUSIÓN

Crear ocasiones pesa más que pasar mucho

XG MEDIO POR PARTIDO



CORRELACIÓN CON LOS PUNTOS



QUÉ QUEREMOS QUE SE LLEVEN

- En las victorias el equipo creó mucho más xG (1,83) que en empates y derrotas (cerca de 0,8). También tiró más: 11,5 tiros frente a 9,4 en derrotas.
- Pasar mucho o pasar bien casi no influye en los puntos.
- Solo hay 38 partidos y correlación no es causa: quien va ganando también juega distinto. Vale cualquier pareja de variables si se justifica con el gráfico.

EJERCICIO 3 · CÓDIGO DE LA SOLUCIÓN

Media por resultado y correlación con los puntos

python · Colab

```
variable = "xg" # prueba también: "tiros", "pases", "acierto_pase"
por_partido.groupby("resultado")[variable].mean().round(2)

columnas = ["puntos", "tiros", "xg", "pases", "acierto_pase"]
por_partido[columnas].corr()["puntos"].round(2)
```

Media por grupo

Compara el valor típico de cada variable en victorias, empates y derrotas.

corr()

Va de −1 a 1: cerca de 0, no hay relación lineal; cerca de 1, crecen juntas.

Con cautela

38 partidos y correlación no es causa: quien va ganando también juega distinto.

EJERCICIO 4 · OPCIONES

¿Qué pasaría si usamos dropna()?

En el notebook ya rellenamos los nulos con «Completo». Para ver el error, los volvemos a poner con replace y comparamos.

A Contar lo que perdemos

Pases antes y después de borrar los nulos de pass_outcome.

```
sin_nulos = pases["pass_outcome"]
  .replace("Completo", np.nan).drop
na()
```

B dropna a toda la tabla

¿Cuántas filas de eventos sobreviven si borramos cualquier fila con algún nulo?

```
len(eventos.dropna())
```

C El acierto, antes y después

Compara el porcentaje de acierto correcto con el que saldría tras el error.

```
mal = sin_nulos.eq("Completo").mean
()
```

EJERCICIO 4 · CONCLUSIÓN

Un nulo puede ser información, no un error

−75 %
de los pases desaparecen: de 36.399 quedan 9.092

75 % → 0 %
porcentaje de acierto, porque solo sobreviven los pases fallados

128.813 → 0
filas de eventos si aplicamos dropna() a toda la tabla

QUÉ QUEREMOS QUE SE LLEVEN

- En pass_outcome un vacío significa pase completo: borrarlo elimina justo los pases buenos.
- Muchos nulos son estructurales: un pase no tiene xG y un tiro no tiene destinatario.
- Antes de borrar o rellenar un nulo, pregunta qué significa. Opciones: conservarlo, rellenarlo con un valor con sentido («Completo») o borrar solo si es un error real.

EJERCICIO 4 · CÓDIGO DE LA SOLUCIÓN

Qué pasa si aplicamos dropna()

python · Colab

```
antes = len(pases)
sin_nulos = pases["pass_outcome"].replace("Completo", np.nan).dropna()
print(antes, len(sin_nulos)) # 36399 -> 9092
mal = sin_nulos.eq("Completo").mean() * 100 # 0 %
len(eventos.dropna()) # 0 filas
```

replace → NaN

Simula el estado anterior a rellenar: los pases completos vuelven a ser nulos.

dropna()

Borra las filas con algún nulo: elimina justo los pases buenos.

Toda la tabla

Con dropna() en eventos no queda ninguna fila: cada evento tiene nulos que no le aplican.

CIERRE

Lo que nos llevamos

- Big Data es volumen, velocidad, variedad y veracidad.
- Un proyecto recorre ingesta, almacenamiento, procesamiento y visualización.
- La calidad del dato y la normativa condicionan todo lo demás.
- Con pandas se carga, limpia y transforma un dataset real.

SIGUIENTE BLOQUE

Spark en Databricks

El mismo dataset, ahora procesado en un clúster.